# LIN7076 – Foundations of Computational Linguistics

## Context-free grammars

Adèle Hénot-Mortier

18/11/2025

Queen Mary University of London

# Plan for today

Can we come up with simple rules summarizing a language or a fragment thereof?

- Last week, we discussed a fundamental limitation of regexes: they **cannot keep track of arbitrary occurrence counts**.
- We have also seen that regexes **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- Today we'll discuss a graphical representation of regexes, **automata**, and show that they run into similar issues.
- We'll then introduce **context-free grammars** to encode more structure and recursivity.

# Plan for today

## Can we come up with simple rules summarizing a language or a fragment thereof?

- Last week, we discussed a fundamental limitation of regexes: they **cannot keep track of arbitrary occurrence counts**.
- We have also seen that regexes **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- Today we'll discuss a graphical representation of regexes, **automata**, and show that they run into similar issues.
- We'll then introduce **context-free grammars** to encode more structure and recursivity.

# Plan for today

## Can we come up with simple rules summarizing a language or a fragment thereof?

- Last week, we discussed a fundamental limitation of regexes: they **cannot keep track of arbitrary occurrence counts**.
- We have also seen that regexes **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- Today we'll discuss a graphical representation of regexes, **automata**, and show that they run into similar issues.
- We'll then introduce **context-free grammars** to encode more structure and recursivity.

# Plan for today

## Can we come up with simple rules summarizing a language or a fragment thereof?

- Last week, we discussed a fundamental limitation of regexes: they **cannot keep track of arbitrary occurrence counts**.
- We have also seen that regexes **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- Today we'll discuss a graphical representation of regexes, **automata**, and show that they run into similar issues.
- We'll then introduce **context-free grammars** to encode more structure and recursivity.

# Plan for today

## Can we come up with simple rules summarizing a language or a fragment thereof?

- Last week, we discussed a fundamental limitation of regexes: they **cannot keep track of arbitrary occurrence counts**.
- We have also seen that regexes **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- Today we'll discuss a graphical representation of regexes, **automata**, and show that they run into similar issues.
- We'll then introduce **context-free grammars** to encode more structure and recursivity.

# Automata

# Graphical representation of regexes: automata

- Regexes can be represented using diagrams called **automata**. The two formalisms are equally expressive.
- Just like regexes, automata express concatenation, union, and the Kleene star operation and can become increasingly complex by combining these atomic operations together.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end).



Initial state



Accepting state

- States are connected with arrows labeled with words of the alphabet $\Sigma$ (including the empty word $\epsilon$).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

# Graphical representation of regexes: automata

- Regexes can be represented using diagrams called **automata**. The two formalisms are equally expressive.
- Just like regexes, automata express concatenation, union, and the Kleene star operation and can become increasingly complex by combining these atomic operations together.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end).



Initial state



Accepting state

- States are connected with arrows labeled with words of the alphabet $\Sigma$ (including the empty word $\epsilon$).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

# Graphical representation of regexes: automata

- Regexes can be represented using diagrams called **automata**. The two formalisms are equally expressive.
- Just like regexes, automata express concatenation, union, and the Kleene star operation and can become increasingly complex by combining these atomic operations together.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end).



Initial state



Accepting state

- States are connected with arrows labeled with words of the alphabet $\Sigma$ (including the empty word $\epsilon$).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

# Graphical representation of regexes: automata

- Regexes can be represented using diagrams called **automata**. The two formalisms are equally expressive.
- Just like regexes, automata express concatenation, union, and the Kleene star operation and can become increasingly complex by combining these atomic operations together.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end).



Initial state



Accepting state

- States are connected with arrows labeled with words of the alphabet $\Sigma$ (including the empty word $\epsilon$).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

# Graphical representation of regexes: automata

- Regexes can be represented using diagrams called **automata**. The two formalisms are equally expressive.
- Just like regexes, automata express concatenation, union, and the Kleene star operation and can become increasingly complex by combining these atomic operations together.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end).



Initial state



Accepting state

- States are connected with arrows labeled with words of the alphabet $\Sigma$ (including the empty word $\epsilon$).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

# Concatenation automaton

- The following automaton accepts only one sentence, *the boy*. It corresponds to the concatenated regex the boy.




- More generally, regex **concatenation** is represented by **arrows between states**.

# Concatenation automaton

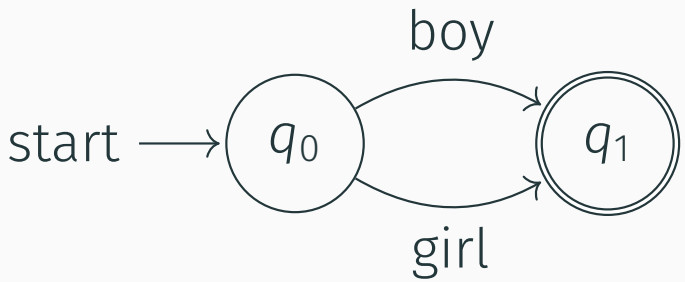
- The following automaton accepts only one sentence, *the boy*. It corresponds to the concatenated regex the boy.




- More generally, regex **concatenation** is represented by **arrows between states**.

# Union automaton

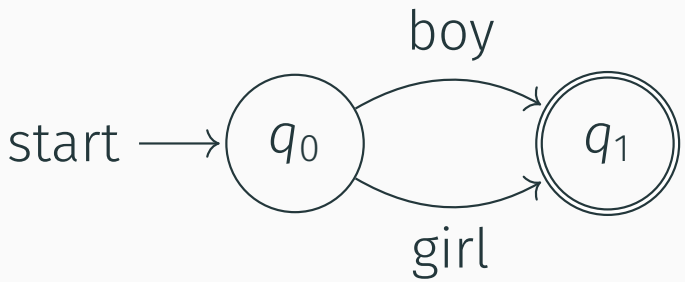
- The following automaton accepts *boy* or *girl*, i.e. it corresponds to the regex union operation (boy|girl).




- More generally, regex **union** is represented by **multiple arrows** originating from the same state.

# Union automaton

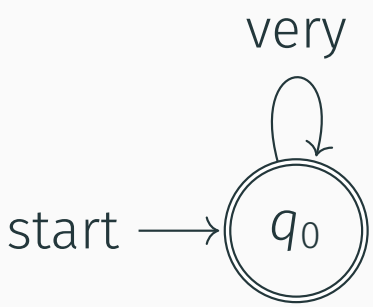
- The following automaton accepts *boy* or *girl*, i.e. it corresponds to the regex union operation (boy|girl).




- More generally, regex **union** is represented by **multiple arrows** originating from the same state.

# Kleene star automaton

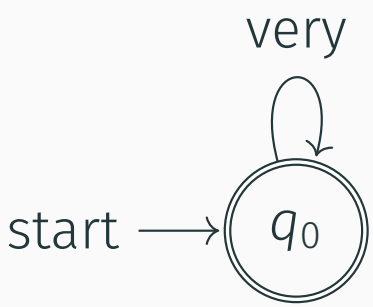
- The following automaton accepts $\epsilon$, *very*, *very very*, *very very very*...i.e. it corresponds to (*very*$^*$).




- More generally, **Kleene star** operations are **feedback loops** to "previous" states.

# Kleene star automaton

- The following automaton accepts $\epsilon$, *very*, *very very*, *very very very*...i.e. it corresponds to (*very*$^*$).




- More generally, **Kleene star** operations are **feedback loops** to "previous" states.

# Practice: automaton for intensified adjectival phrases

- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|$\epsilon$)(boy|girl).

# Practice: automaton for intensified adjectival phrases

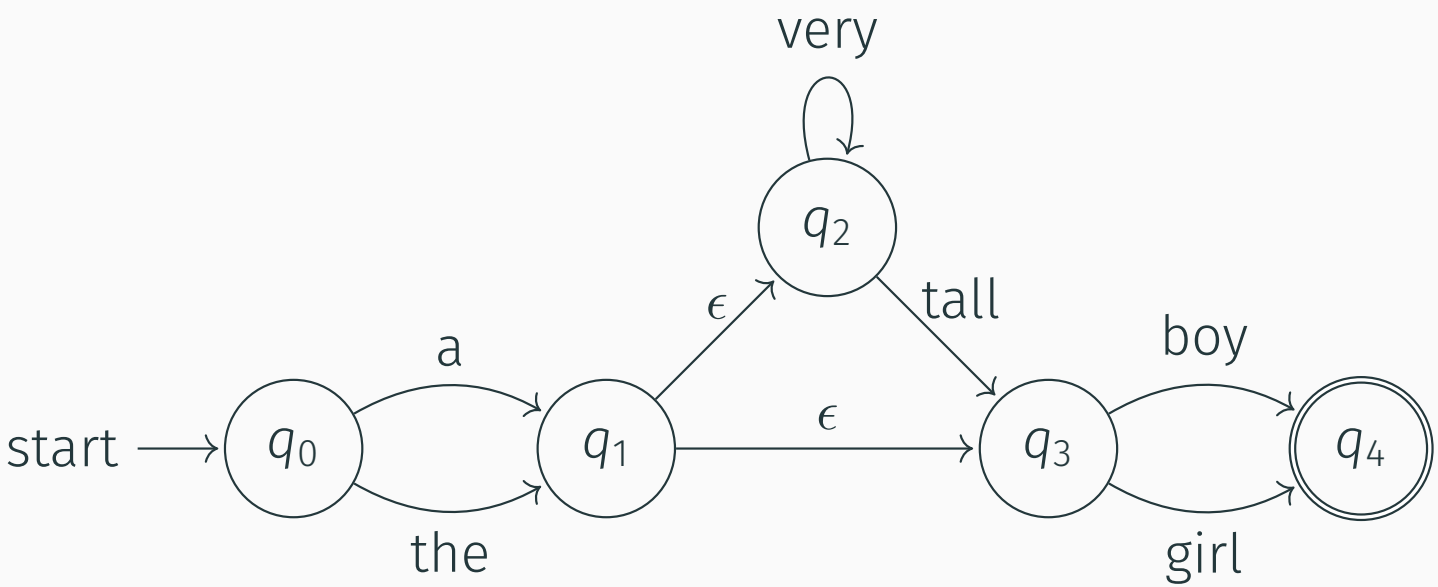
- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|$\epsilon$)(boy|girl).
- It meant that first we want a determiner (*a* or *the*), then we want either an optionally intensified adjective (*very very ... very tall*) or nothing, and lastly, we want a noun (*boy* or *girl*).

# Practice: automaton for intensified adjectival phrases

- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|$\epsilon$)(boy|girl).
- It meant that first we want a determiner (*a* or *the*), then we want either an optionally intensified adjective (*very very ... very tall*) or nothing, and lastly, we want a noun (*boy* or *girl*).
- What's the automaton for this?

# Practice: automaton for intensified adjectival phrases

- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|$\epsilon$)(boy|girl).
- It meant that first we want a determiner (*a* or *the*), then we want either an optionally intensified adjective (*very very ... very tall*) or nothing, and lastly, we want a noun (*boy* or *girl*).
- What's the automaton for this?



# Practice: automaton for right-embedded complementation

- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).

# Practice: automaton for right-embedded complementation

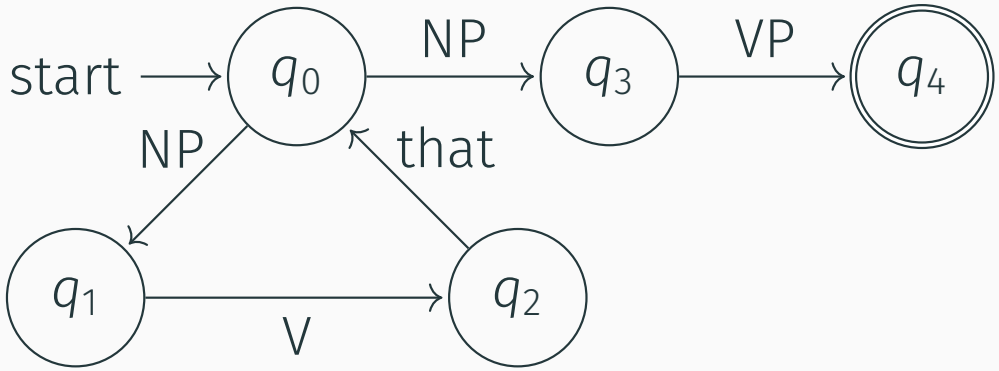
- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).
- It meant that first we want NP V that (an embedding sentence) zero or more times, and then we finish by NP VP (the last embedded sentence).

# Practice: automaton for right-embedded complementation

- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).
- It meant that first we want NP V that (an embedding sentence) zero or more times, and then we finish by NP VP (the last embedded sentence).
- What's the automaton for this?

# Practice: automaton for right-embedded complementation

- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).
- It meant that first we want NP V that (an embedding sentence) zero or more times, and then we finish by NP VP (the last embedded sentence).
- What's the automaton for this?



# Some extra challenges

# One more challenge

- Last week, we saw that regexes struggle with center-embedded structures, and ambiguous structures like *tall boys and girls*.
- Here is another problematic example.

(1) The photographer saw the bird on the alligator with long legs.

# Practice: automaton for PP-attachment

(2) The photographer saw the bird on the alligator with long legs.

- Suppose we are dealing with a very coarse alphabet $\Sigma = \{NP, V, PP\}$. A regex for sentences like (2) is

# Practice: automaton for PP-attachment

(3) The photographer saw the bird on the alligator with long legs.

- Suppose we are dealing with a very coarse alphabet $\Sigma = \{NP, V, PP\}$. A regex for sentences like (2) is NP V NP (PP)*.

# Practice: automaton for PP-attachment

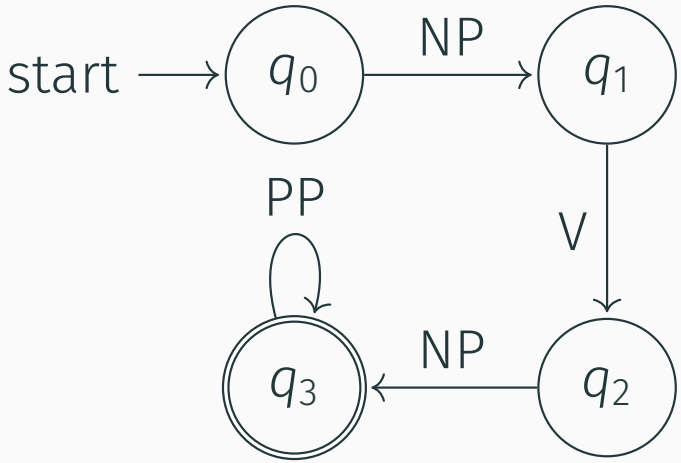
(4) The photographer saw the bird on the alligator with long legs.

- Suppose we are dealing with a very coarse alphabet $\Sigma = \{NP, V, PP\}$. A regex for sentences like (2) is NP V NP (PP)*.
- What's the automaton for this?

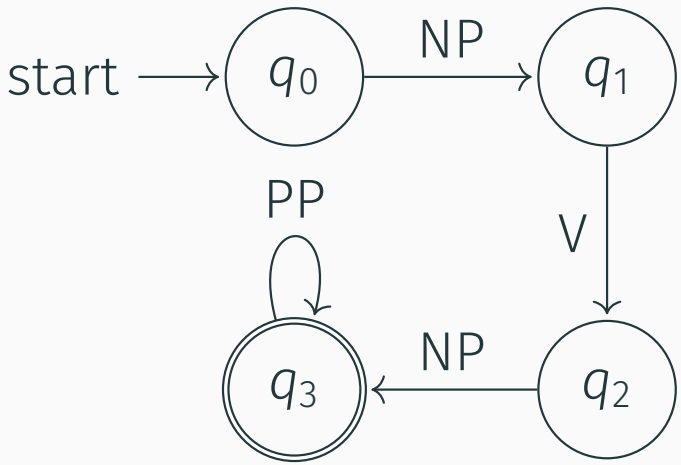
# Practice: automaton for PP-attachment

(5) The photographer saw the bird on the alligator with long legs.

- Suppose we are dealing with a very coarse alphabet $\Sigma = \{NP, V, PP\}$. A regex for sentences like (2) is NP V NP (PP)*.
- What's the automaton for this?



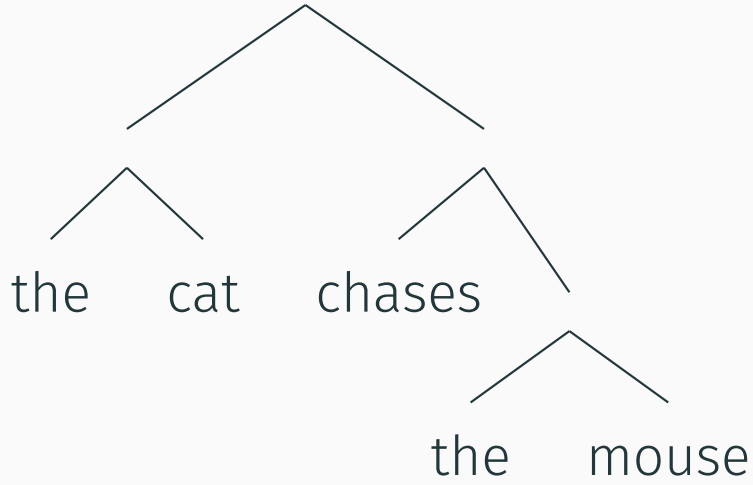
# The challenge of PPs




- The above automaton just says that a sentence like (2) can end with 0 or more PPs.
- It does not say anything about what the PPs modify!
- And they are even less capable of cashing out the constraint which says that if the linearly last PP modifies the object NP, then any PP before it must too (if the bird has long legs, then the bird, and not the photographer, is on the alligator!)

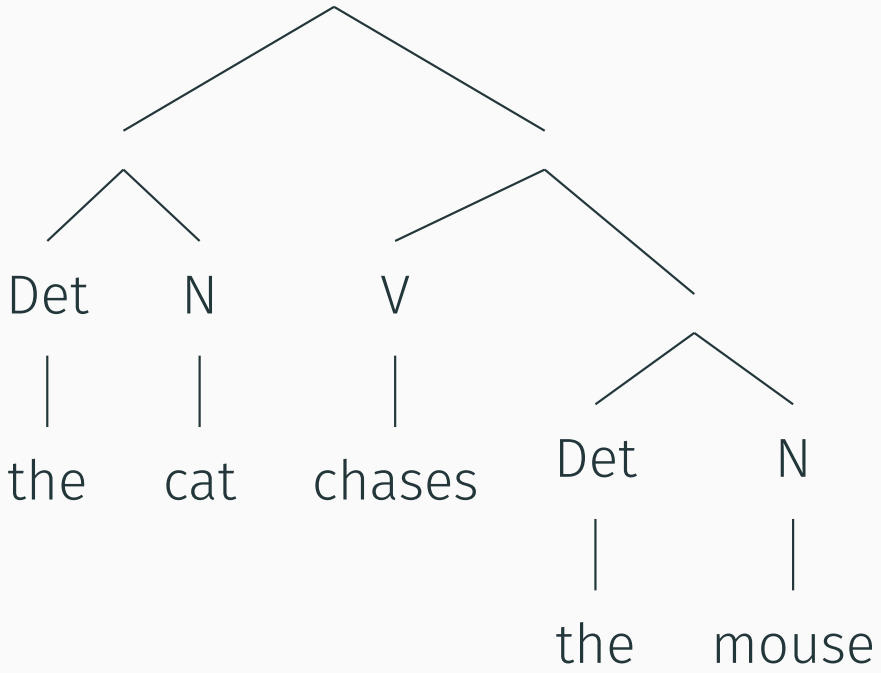
# Context-free grammars

# Intuitive Syntax trees



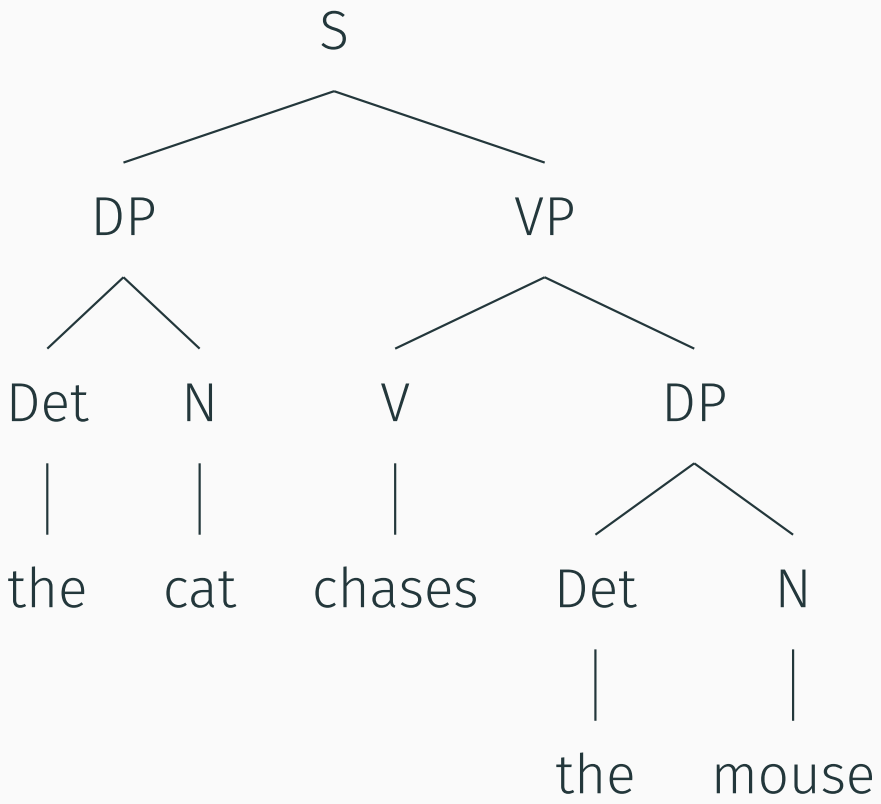

- Iteratively merging nodes together based on intuitions about what forms a unit (constituent).

# Intuitive Syntax trees




- Adding Part-of-Speech labels to the lexical items.

# Intuitive Syntax trees




- Adding labels to intermediate nodes according to how they "behave": DP=Determiner Phrase; VP=Verb Phrase, S=Sentence, NP=Noun Phrase, PP=Preposition Phrase...

# Generating syntax trees

- One can use context-free grammar to generate syntax trees. A context-free grammar is defined as:
  - A finite set $\Sigma$ of **terminals**, (leaves, i.e. actual words of the language).
  - A finite set $V$ of **nonterminals** (intermediate nodes, i.e. phrase or category labels). $V$ contains a special nonterminal S used to represent entire sentences (root).
  - A set $R$ of **production rules**, i.e. input $\rightarrow$ output pairs, where inputs are single nonterminals (understood as mother nodes) and outputs are (non)terminals or concatenations thereof (understood as children node(s)). For instance, 

corresponds to the production rule S $\rightarrow$ NP VP
- Pāṇini, a scholar in ancient India, came up with a grammar of Sanskrit following a very similar formalism in the mid-1st millennium BCE.

# Generating syntax trees

- One can use context-free grammar to generate syntax trees. A context-free grammar is defined as:
  - A finite set $\Sigma$ of **terminals**, (leaves, i.e. actual words of the language).
  - A finite set $V$ of **nonterminals** (intermediate nodes, i.e. phrase or category labels). $V$ contains a special nonterminal S used to represent entire sentences (root).
  - A set $R$ of **production rules**, i.e. input $\rightarrow$ output pairs, where inputs are single nonterminals (understood as mother nodes) and outputs are (non)terminals or concatenations thereof (understood as children node(s)). For instance, 

corresponds to the production rule S $\rightarrow$ NP VP
- Pāṇini, a scholar in ancient India, came up with a grammar of Sanskrit following a very similar formalism in the mid-1st millennium BCE.

# Generating syntax trees

- One can use context-free grammar to generate syntax trees. A context-free grammar is defined as:
  - A finite set $\Sigma$ of **terminals**, (leaves, i.e. actual words of the language).
  - A finite set $V$ of **nonterminals** (intermediate nodes, i.e. phrase or category labels). $V$ contains a special nonterminal S used to represent entire sentences (root).
  - A set $R$ of **production rules**, i.e. input $\rightarrow$ output pairs, where inputs are single nonterminals (understood as mother nodes) and outputs are (non)terminals or concatenations thereof (understood as children node(s)). For instance, 

corresponds to the production rule S $\rightarrow$ NP VP
- Pāṇini, a scholar in ancient India, came up with a grammar of Sanskrit following a very similar formalism in the mid-1st millennium BCE.

# Generating syntax trees

- One can use context-free grammar to generate syntax trees. A context-free grammar is defined as:
  - A finite set $\Sigma$ of **terminals**, (leaves, i.e. actual words of the language).
  - A finite set $V$ of **nonterminals** (intermediate nodes, i.e. phrase or category labels). $V$ contains a special nonterminal S used to represent entire sentences (root).
  - A set $R$ of **production rules**, i.e. input $\rightarrow$ output pairs, where inputs are single nonterminals (understood as mother nodes) and outputs are (non)terminals or concatenations thereof (understood as children node(s)). For instance, 

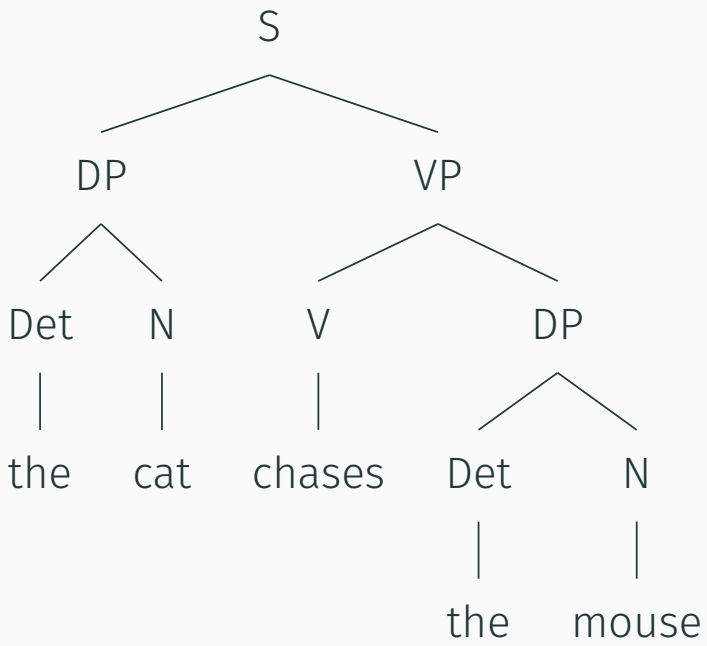
corresponds to the production rule S $\rightarrow$ NP VP
- Pāṇini, a scholar in ancient India, came up with a grammar of Sanskrit following a very similar formalism in the mid-1st millennium BCE.

# Generating syntax trees

- One can use context-free grammar to generate syntax trees. A context-free grammar is defined as:
  - A finite set $\Sigma$ of **terminals**, (leaves, i.e. actual words of the language).
  - A finite set $V$ of **nonterminals** (intermediate nodes, i.e. phrase or category labels). $V$ contains a special nonterminal S used to represent entire sentences (root).
  - A set $R$ of **production rules**, i.e. input $\rightarrow$ output pairs, where inputs are single nonterminals (understood as mother nodes) and outputs are (non)terminals or concatenations thereof (understood as children node(s)). For instance, 

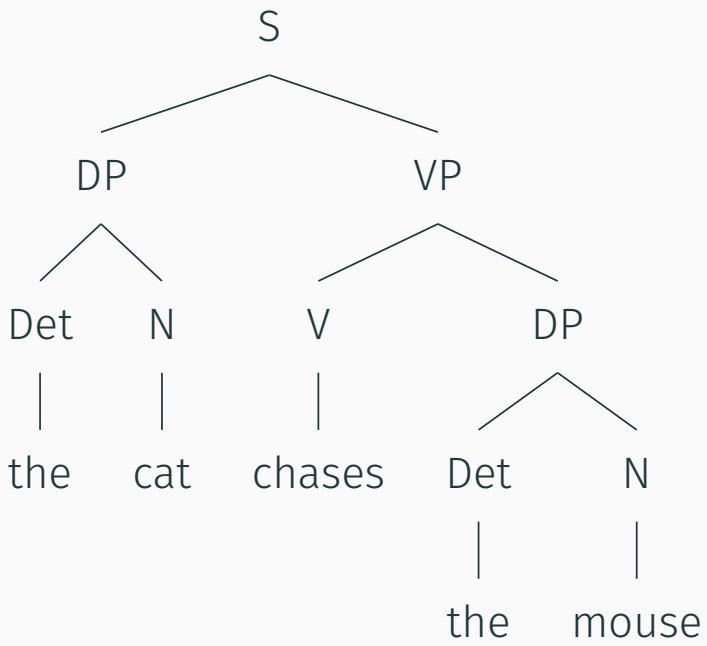
corresponds to the production rule S $\rightarrow$ NP VP
- Pāṇini, a scholar in ancient India, came up with a grammar of Sanskrit following a very similar formalism in the mid-1st millennium BCE.

# Production rules practice



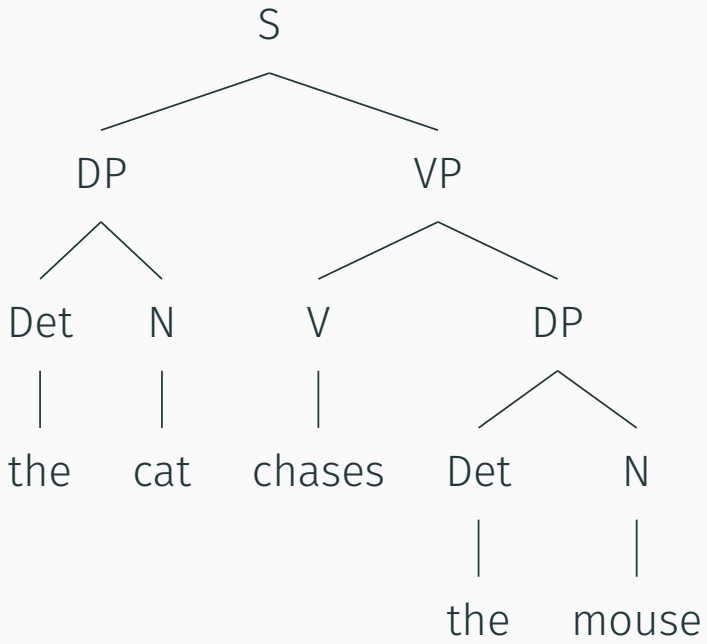

- What are the terminals?

# Production rules practice



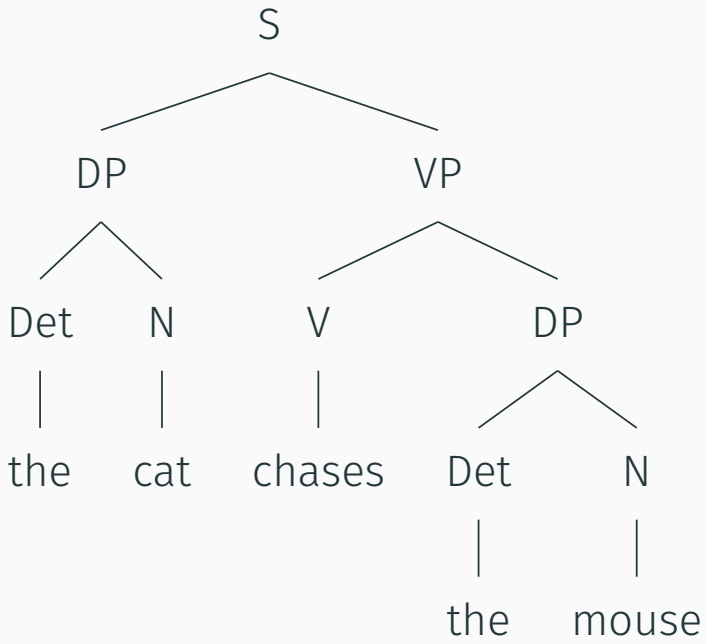

- What are the terminals? *the*, *cat*, *chases*, *mouse*.
- What are the nonterminals?

# Production rules practice



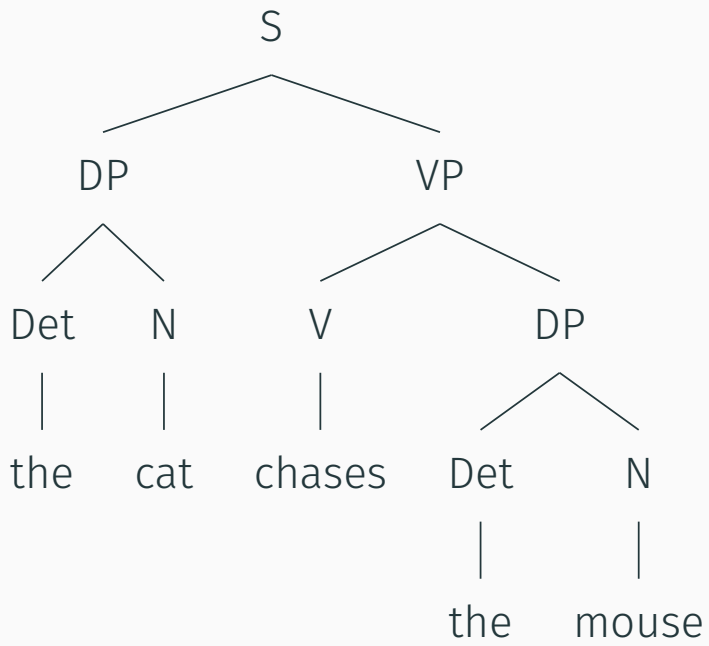

- What are the terminals? *the*, *cat*, *chases*, *mouse*.
- What are the nonterminals? S, DP, VP, Det, N, V.
- What are the production rules?

# Production rules practice




- What are the terminals? *the*, *cat*, *chases*, *mouse*.
- What are the nonterminals? S, DP, VP, Det, N, V.
- What are the production rules? S $\rightarrow$ DP VP; DP $\rightarrow$ Det N; VP $\rightarrow$ V NP; Det $\rightarrow$ the; N $\rightarrow$ cat; N $\rightarrow$ mouse; V $\rightarrow$ chases.
- What else can we generate from those rules?

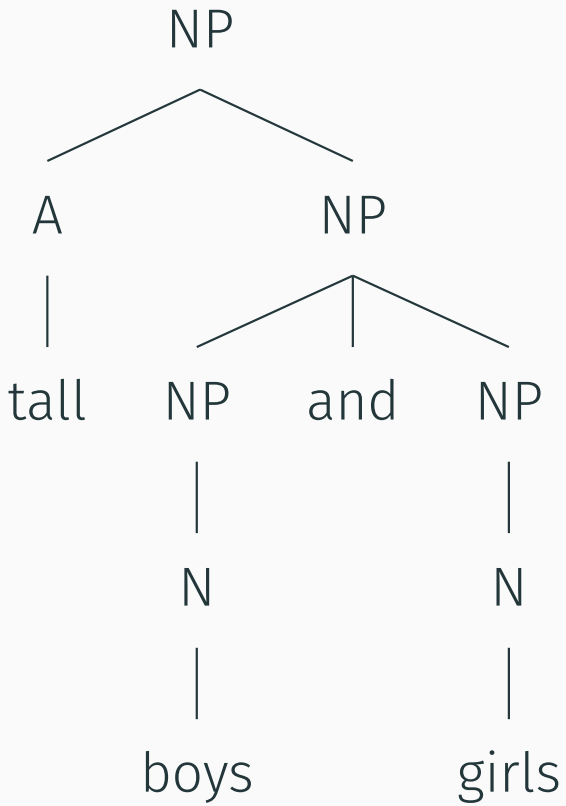
# Production rules practice

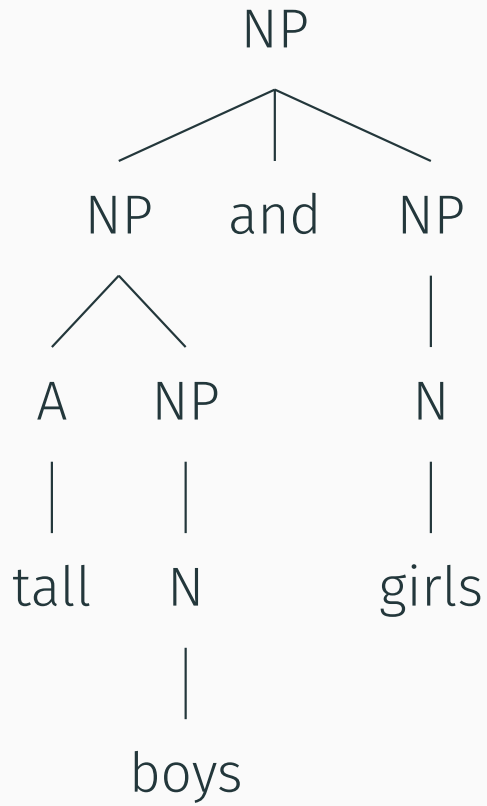



- What are the terminals? *the*, *cat*, *chases*, *mouse*.
- What are the nonterminals? S, DP, VP, Det, N, V.
- What are the production rules? S $\rightarrow$ DP VP; DP $\rightarrow$ Det N; VP $\rightarrow$ V NP; Det $\rightarrow$ the; N $\rightarrow$ cat; N $\rightarrow$ mouse; V $\rightarrow$ chases.
- What else can we generate from those rules? *The mouse chases the cat*.

# Structural ambiguity 1: adjective attachment

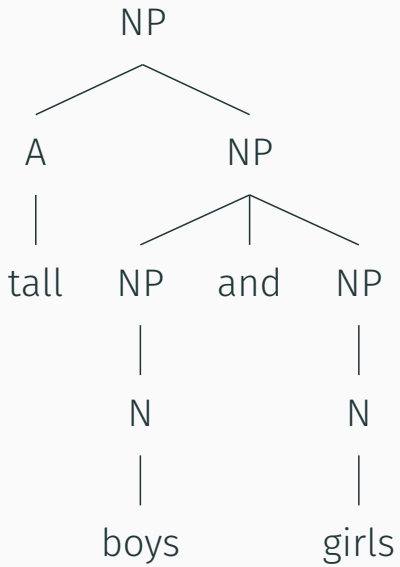
- Intuitively, which tree corresponds to which interpretation?

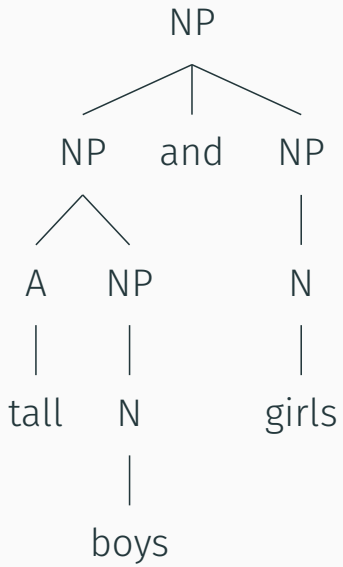



- Terminals? Nonterminals? Production rules?

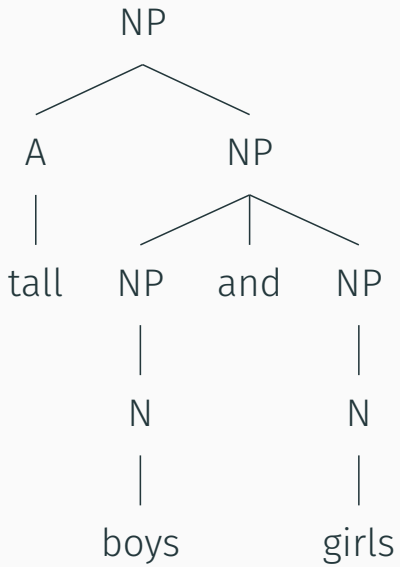
# Structural ambiguity 1: adjective attachment

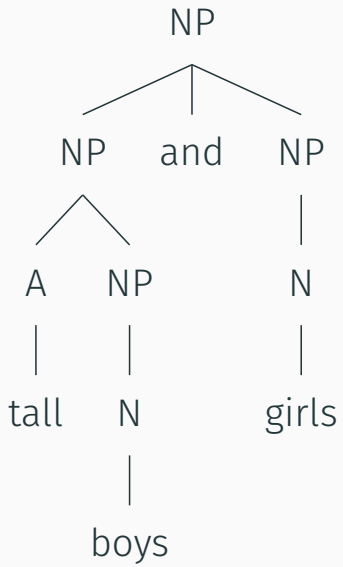



- Terminals? *tall*, *boys*, *and*, *girls*.
- Nonterminals? NP, A, N.
- Production rules? NP $\rightarrow$ A NP; NP $\rightarrow$ NP and NP; NP $\rightarrow$ N.
- What else can we generate? *Boys and tall girls*, *Tall girls and boys*, *Tall tall tall boys and girls*, *Tall boys and tall girls*, *Boys and boys and girls*... Slight **overgeneration issues**!

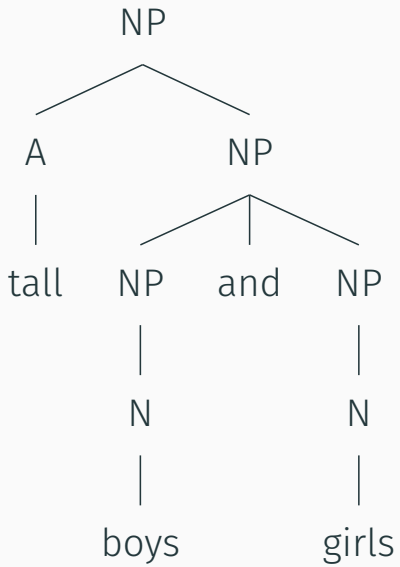
# Structural ambiguity 1: adjective attachment

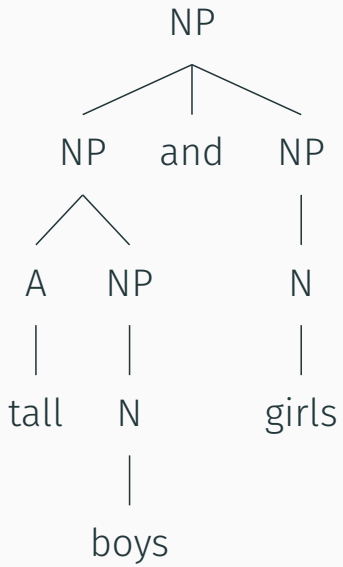



- Terminals? *tall*, *boys*, *and*, *girls*.
- Nonterminals? NP, A, N.
- Production rules? NP $\rightarrow$ A NP; NP $\rightarrow$ NP and NP; NP $\rightarrow$ N.
- What else can we generate? *Boys and tall girls*, *Tall girls and boys*, *Tall tall tall boys and girls*, *Tall boys and tall girls*, *Boys and boys and girls*... Slight **overgeneration issues**!

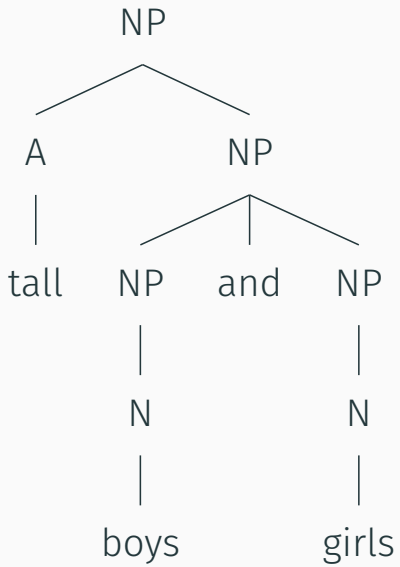
# Structural ambiguity 1: adjective attachment

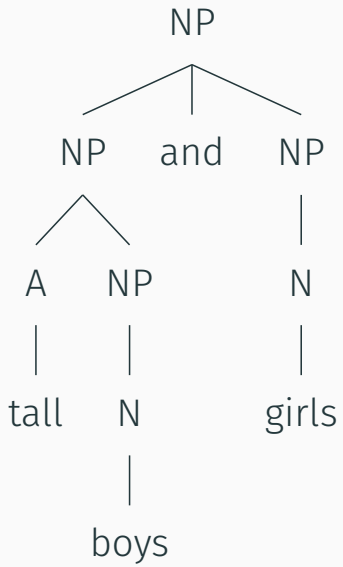



- Terminals? *tall*, *boys*, *and*, *girls*.
- Nonterminals? NP, A, N.
- Production rules? NP $\rightarrow$ A NP; NP $\rightarrow$ NP and NP; NP $\rightarrow$ N.
- What else can we generate? *Boys and tall girls*, *Tall girls and boys*, *Tall tall tall boys and girls*, *Tall boys and tall girls*, *Boys and boys and girls*... Slight **overgeneration issues**!

# Structural ambiguity 1: adjective attachment

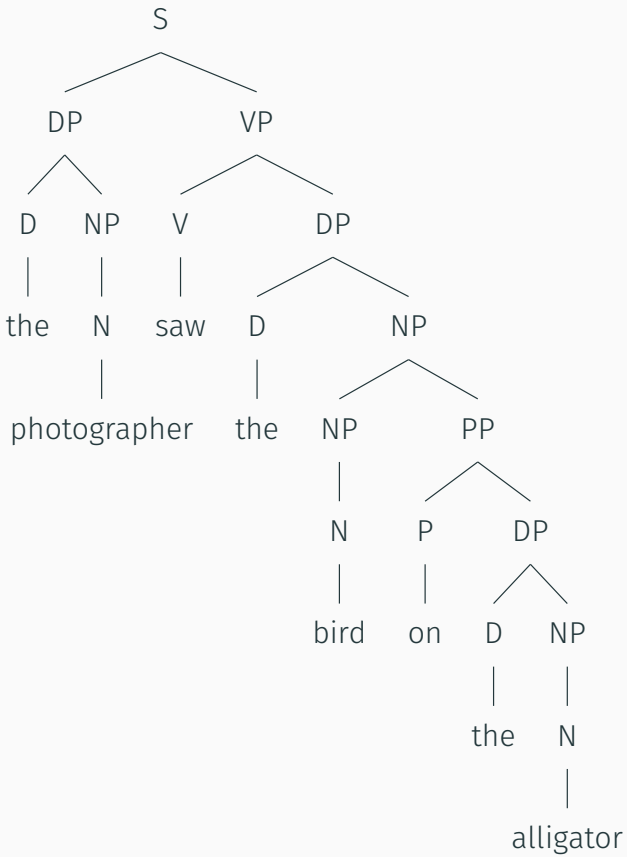


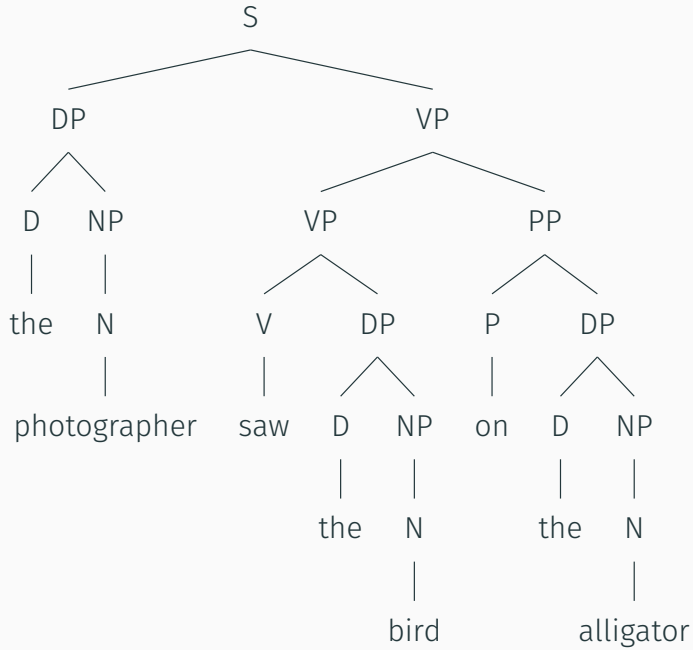




- Terminals? *tall*, *boys*, *and*, *girls*.
- Nonterminals? NP, A, N.
- Production rules? NP $\rightarrow$ A NP; NP $\rightarrow$ NP and NP; NP $\rightarrow$ N.
- What else can we generate? *Boys and tall girls*, *Tall girls and boys*, *Tall tall tall boys and girls*, *Tall boys and tall girls*, *Boys and boys and girls*... Slight **overgeneration issues**!

# Structural ambiguity 2: PP-attachment

(2) The photographer saw the bird on the alligator.




- Which tree intuitively yields which reading?

# Structural ambiguity 2: PP-attachment

- Consider the grammar below. Which reading(s) does it capture?

S $\rightarrow$ DP VP
DP $\rightarrow$ D NP
VP $\rightarrow$ V DP
NP $\rightarrow$ NP PP
NP $\rightarrow$ N

PP $\rightarrow$ P DP
D $\rightarrow$ the
N $\rightarrow$ photographer, bird, alligator
V $\rightarrow$ saw
P $\rightarrow$ on

# Structural ambiguity 2: PP-attachment

- Consider the grammar below. Which reading(s) does it capture?

S $\rightarrow$ DP VP
DP $\rightarrow$ D NP
VP $\rightarrow$ V DP
NP $\rightarrow$ NP PP
NP $\rightarrow$ N

PP $\rightarrow$ P DP
D $\rightarrow$ the
N $\rightarrow$ photographer, bird, alligator
V $\rightarrow$ saw
P $\rightarrow$ on

- The only rule in which PP is an output (daughter node) is NP $\rightarrow$ NP PP. In that rule, PP's sister/mother node is NP, which means PP attaches to an NP. It models the *bird-on-alligator*-reading.

# Structural ambiguity 2: PP-attachment

- Consider the grammar below. Which reading(s) does it capture?

S $\rightarrow$ DP VP
DP $\rightarrow$ D NP
VP $\rightarrow$ V DP
VP $\rightarrow$ VP PP
NP $\rightarrow$ N

PP $\rightarrow$ P DP
D $\rightarrow$ the
N $\rightarrow$ photographer, bird, hill
V $\rightarrow$ saw
P $\rightarrow$ on

# Structural ambiguity 2: PP-attachment

- Consider the grammar below. Which reading(s) does it capture?

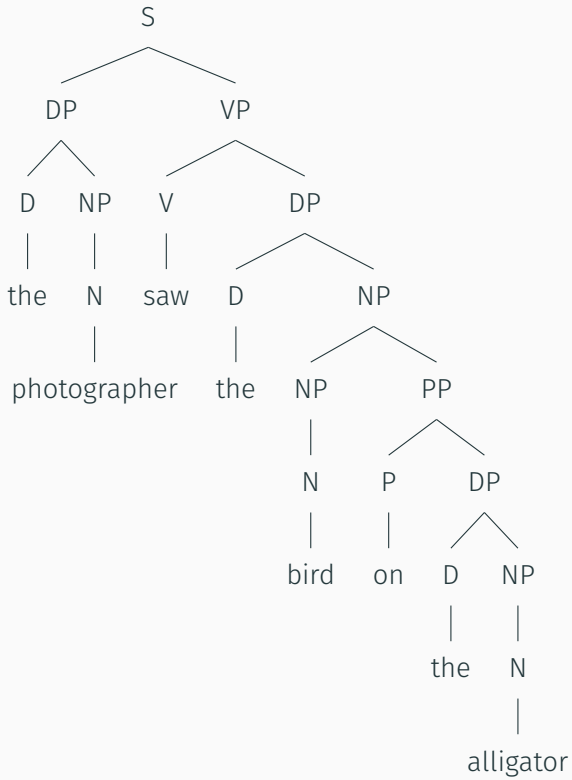
S $\rightarrow$ DP VP
DP $\rightarrow$ D NP
VP $\rightarrow$ V DP
VP $\rightarrow$ VP PP
NP $\rightarrow$ N

PP $\rightarrow$ P DP
D $\rightarrow$ the
N $\rightarrow$ photographer, bird, hill
V $\rightarrow$ saw
P $\rightarrow$ on

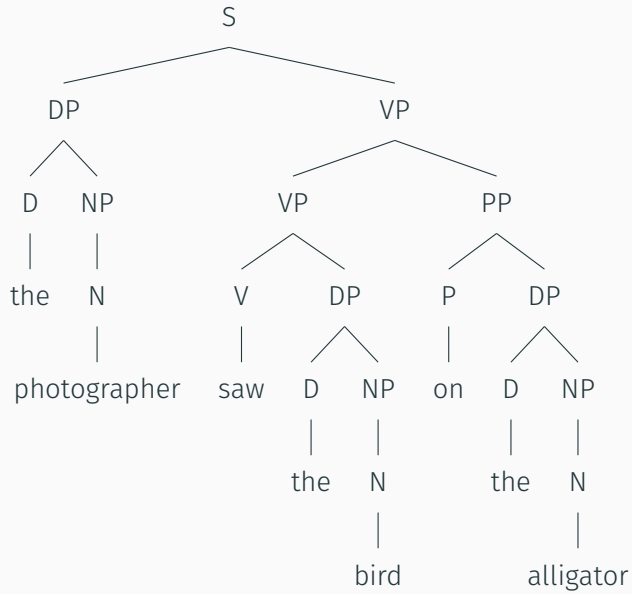
- The only rule in which PP is an output (daughter node) is VP $\rightarrow$ VP PP. In that rule, PP's sister/mother node is VP, which means PP attaches to a VP. It models the *photographer-on-alligator*-reading.

# Structural ambiguity 2': double PP-attachment

(2) The photographer saw the bird on the alligator with long legs.




*Bird-on-alligator* tree

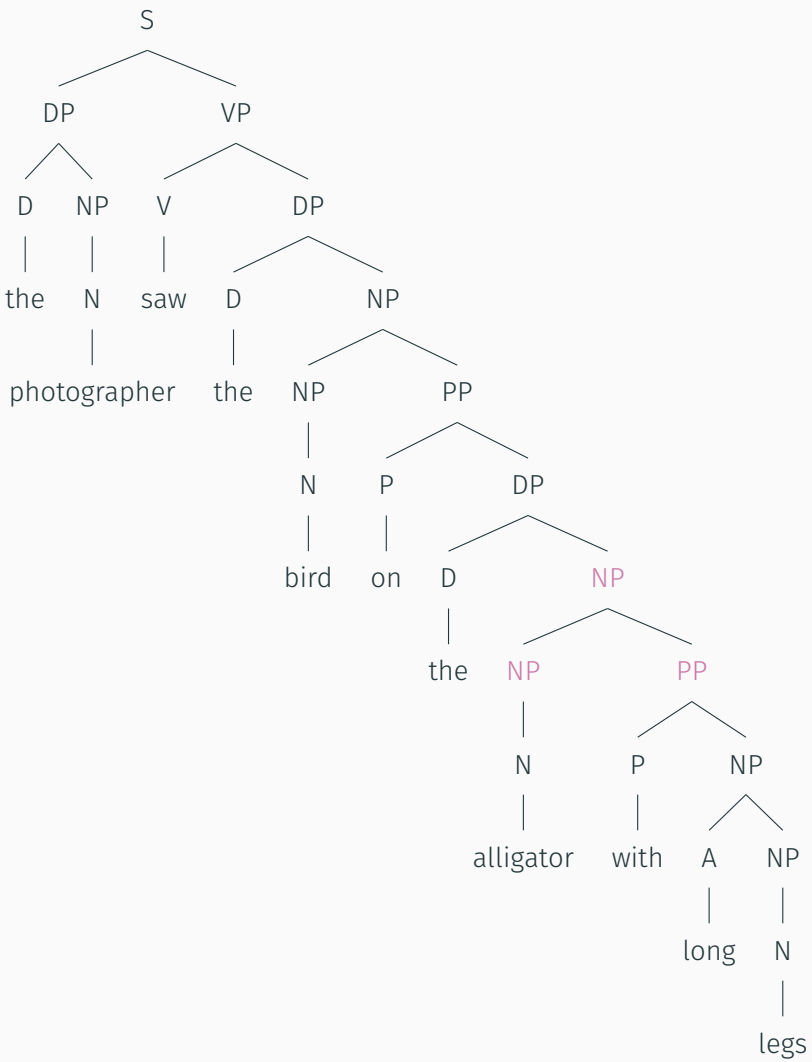



*Photographer-on-alligator* tree

- Two possible PP-introducing rules: VP $\rightarrow$ VP PP and NP $\rightarrow$ NP PP.
- Where can we "insert" the PP *with long legs* in the above trees? Which attachment site yields which reading?

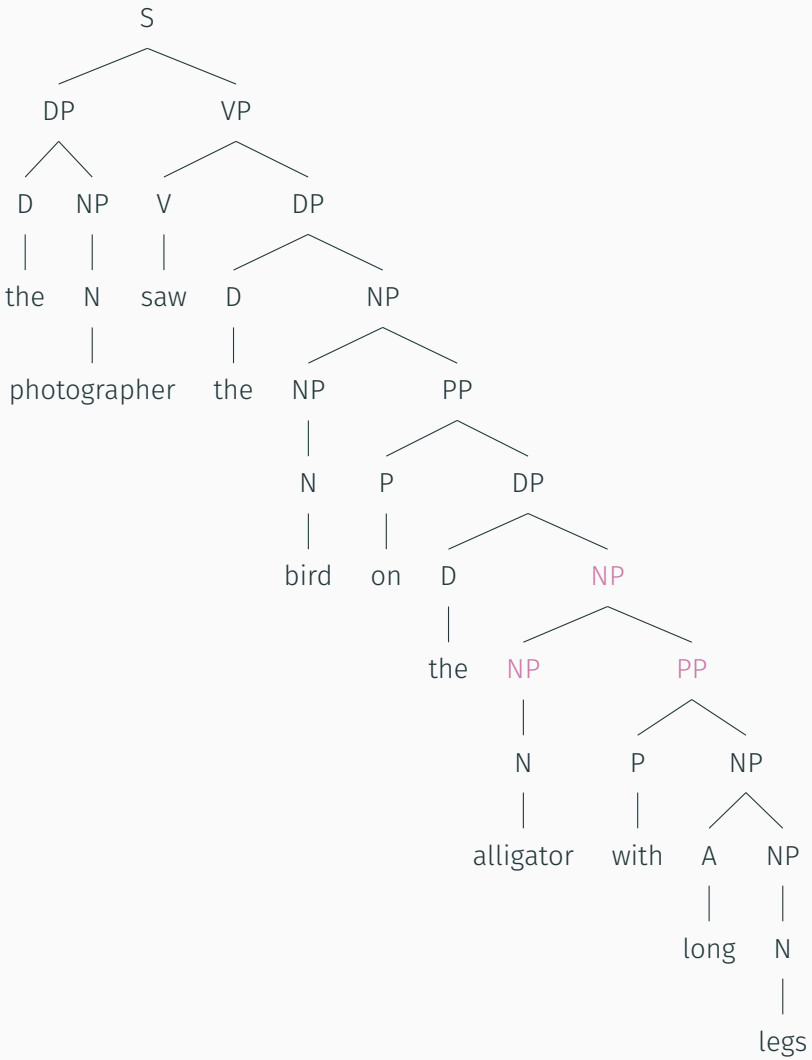
# Structural ambiguity 2': double PP-attachment

(2) The photographer saw the bird on the alligator with long legs.



# Structural ambiguity 2': double PP-attachment

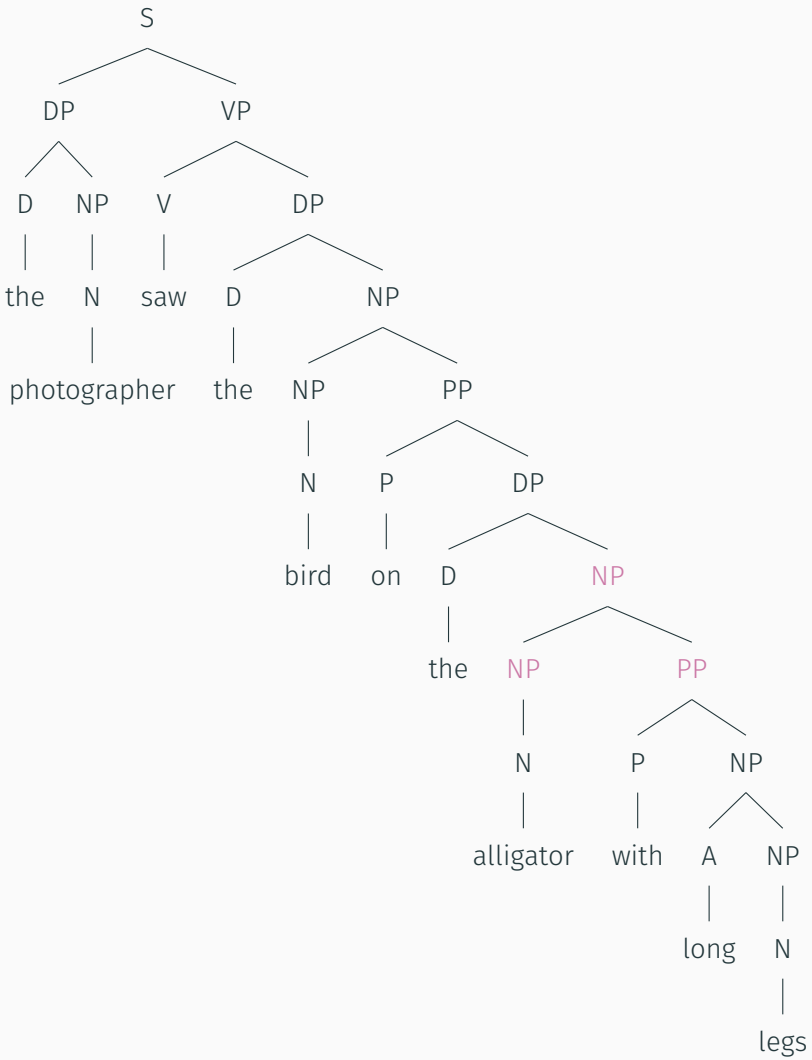
(2) The photographer saw the bird on the alligator with long legs.




- We add a PP $\rightarrow$ P NP rule (to get *with legs*), and a NP $\rightarrow$ rule (to get *long legs*).

# Structural ambiguity 2': double PP-attachment

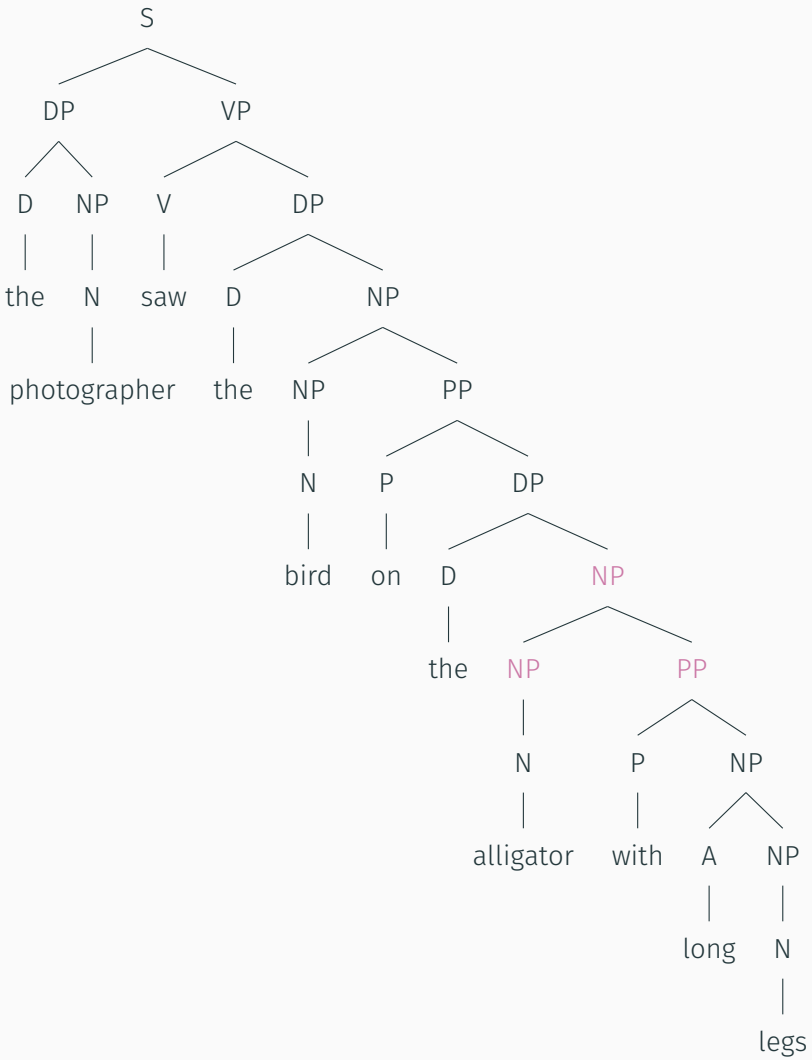
(2) The photographer saw the bird on the alligator with long legs.




- We add a PP $\rightarrow$ P NP rule (to get *with legs*), and a NP $\rightarrow$ rule (to get *long legs)*.
- We modify the bird-on-alligator tree by applying an extra NP $\rightarrow$ NP PP rule right above *alligator*, to modify *alligator* with *with long legs*.

# Structural ambiguity 2': double PP-attachment

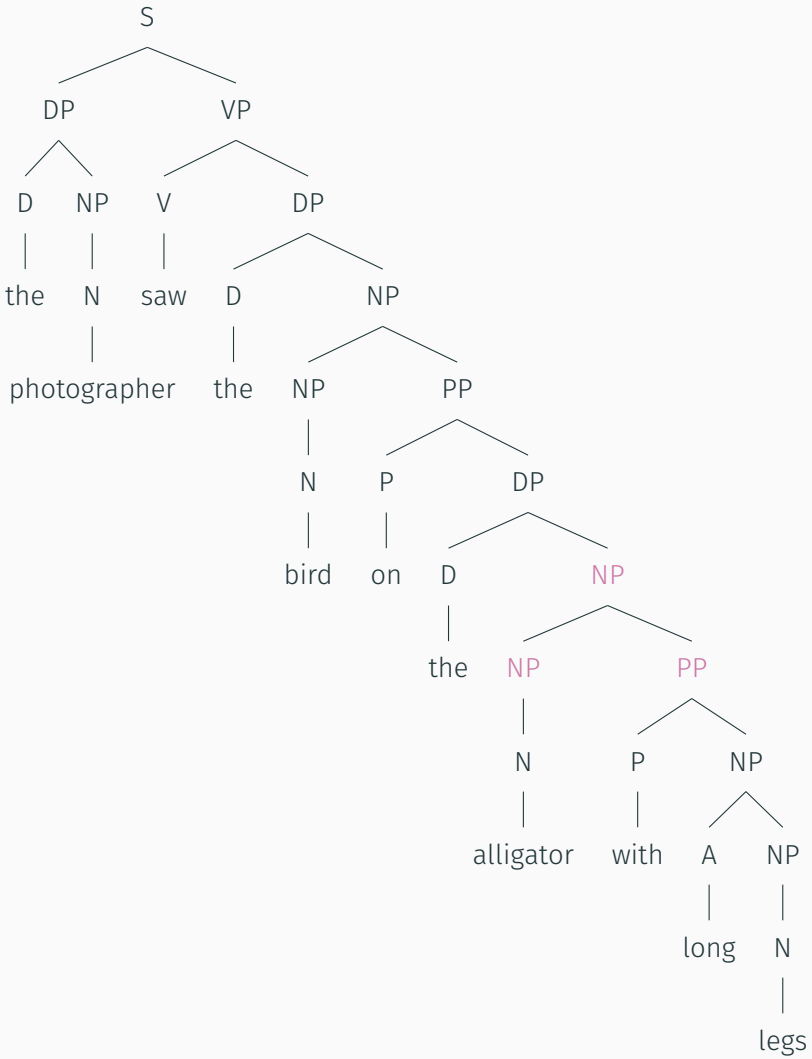
(2) The photographer saw the bird on the alligator with long legs.




- We add a PP $\rightarrow$ P NP rule (to get *with legs*), and a NP $\rightarrow$ rule (to get *long legs)*.
- We modify the bird-on-alligator tree by applying an extra NP $\rightarrow$ NP PP rule right above *alligator*, to modify *alligator* with *with long legs*.
- Reading:

# Structural ambiguity 2': double PP-attachment

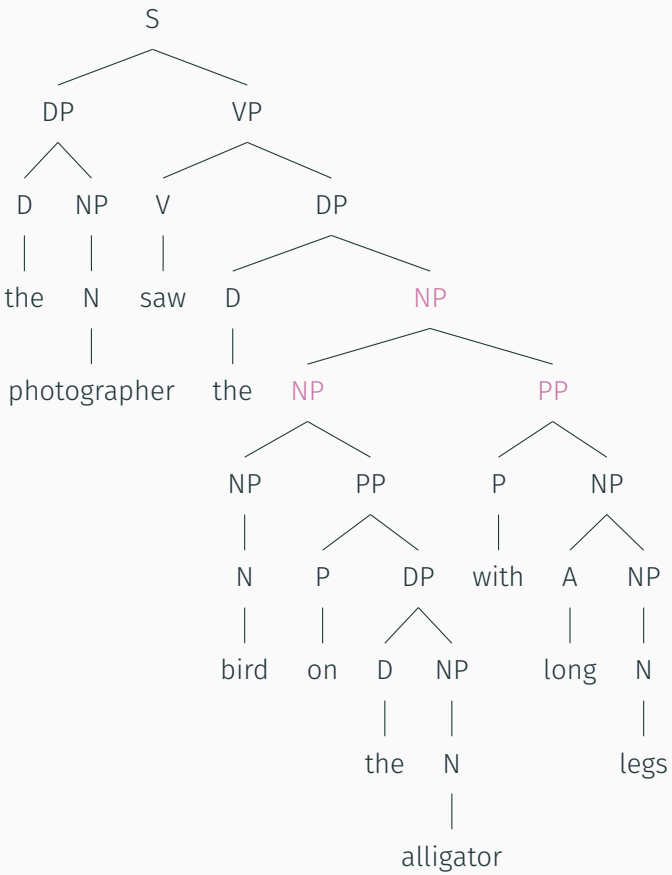
(2) The photographer saw the bird on the alligator with long legs.




- We add a PP $\rightarrow$ P NP rule (to get *with legs*), and a NP $\rightarrow$ rule (to get *long legs)*.
- We modify the bird-on-alligator tree by applying an extra NP $\rightarrow$ NP PP rule right above *alligator*, to modify *alligator* with *with long legs*.
- Reading: The photographer saw a bird that is on an alligator and the alligator has long legs.

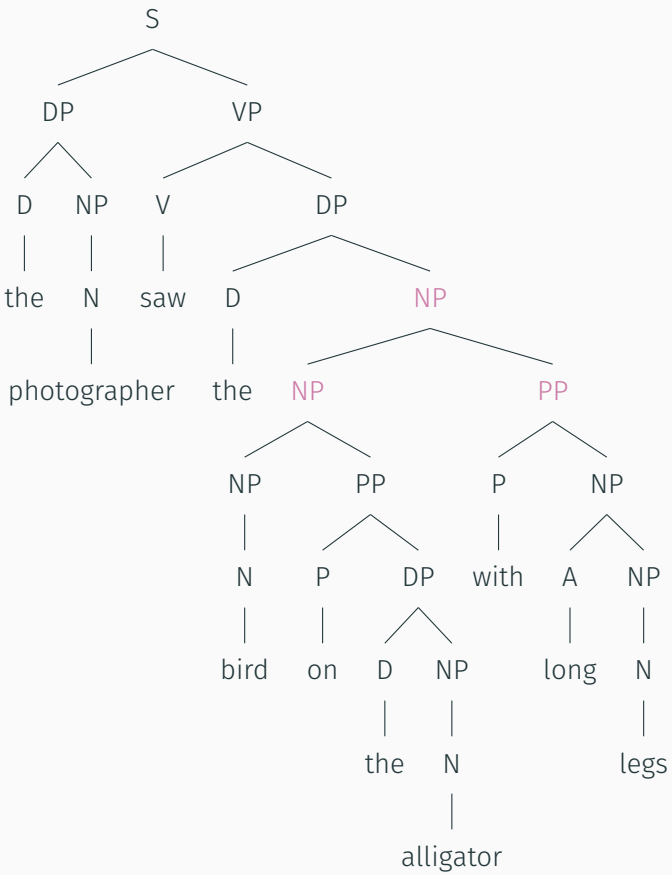
# Structural ambiguity 2': double PP-attachment

(2) The photographer saw the bird on the alligator with long legs.



# Structural ambiguity 2': double PP-attachment

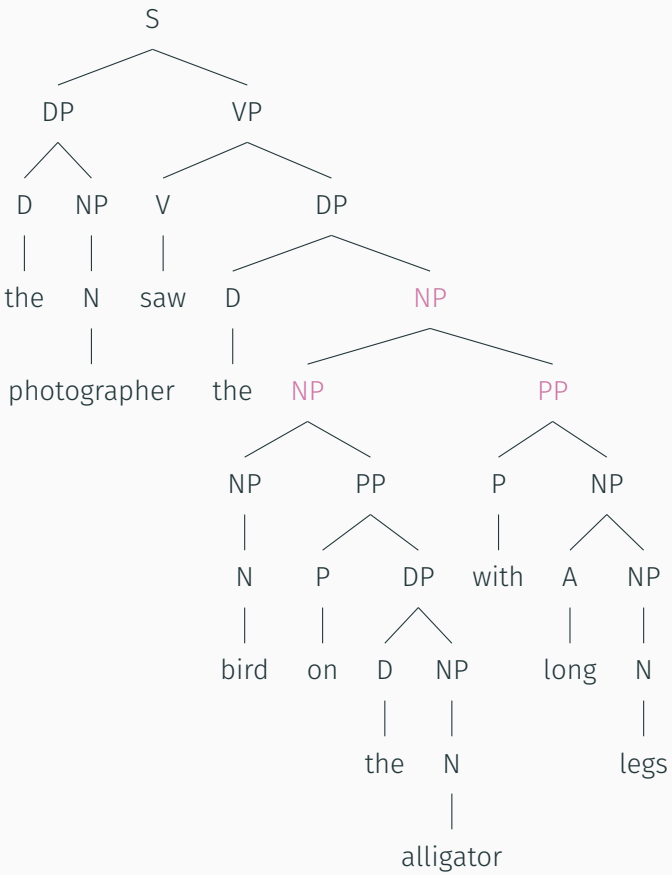
(2) The photographer saw the bird on the alligator with long legs.




- We modify the bird-on-alligator tree by applying an extra NP → NP PP rule right above *bird*, to modify *bird* with *with long legs*.

# Structural ambiguity 2': double PP-attachment

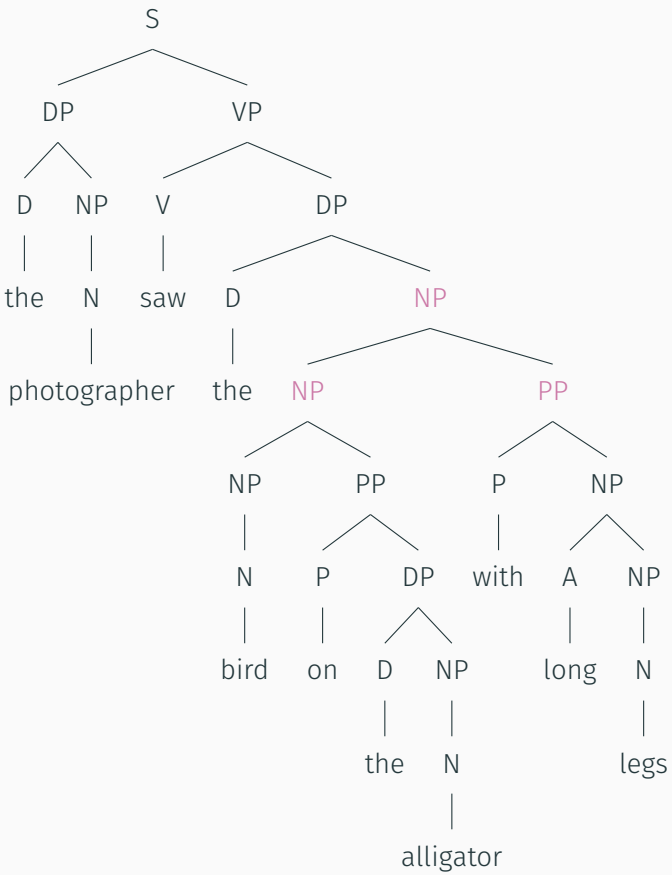
(2) The photographer saw the bird on the alligator with long legs.




- We modify the bird-on-alligator tree by applying an extra NP → NP PP rule right above *bird*, to modify *bird* with *with long legs*.
- Reading:

# Structural ambiguity 2': double PP-attachment

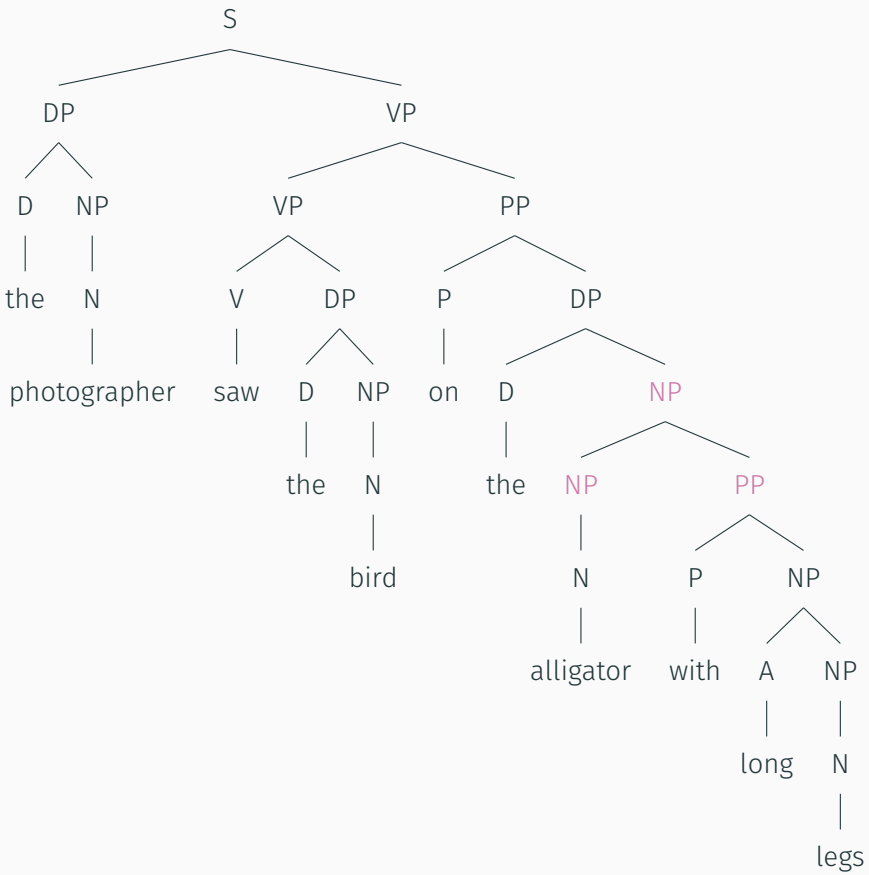
(2) The photographer saw the bird on the alligator with long legs.




- We modify the bird-on-alligator tree by applying an extra NP → NP PP rule right above *bird*, to modify *bird* with *with long legs*.
- Reading: The photographer saw a bird that is on an alligator and that has long legs.

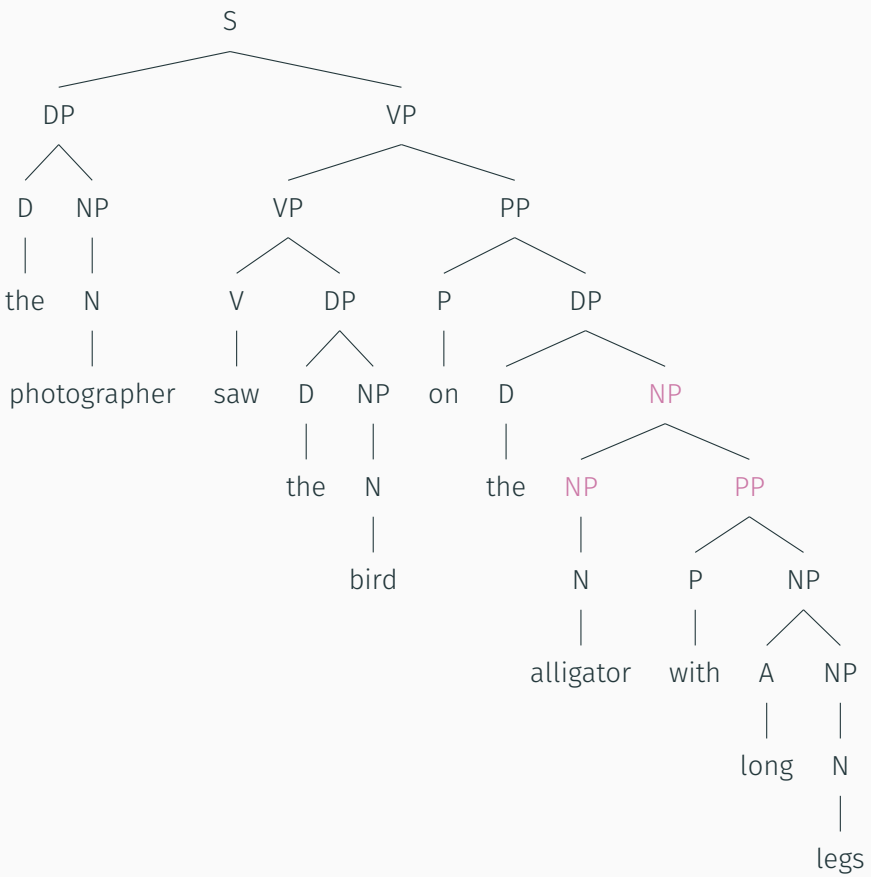
# Structural ambiguity 2': double PP-attachment

(2) The photographer saw the bird on the alligator with long legs.



# Structural ambiguity 2': double PP-attachment

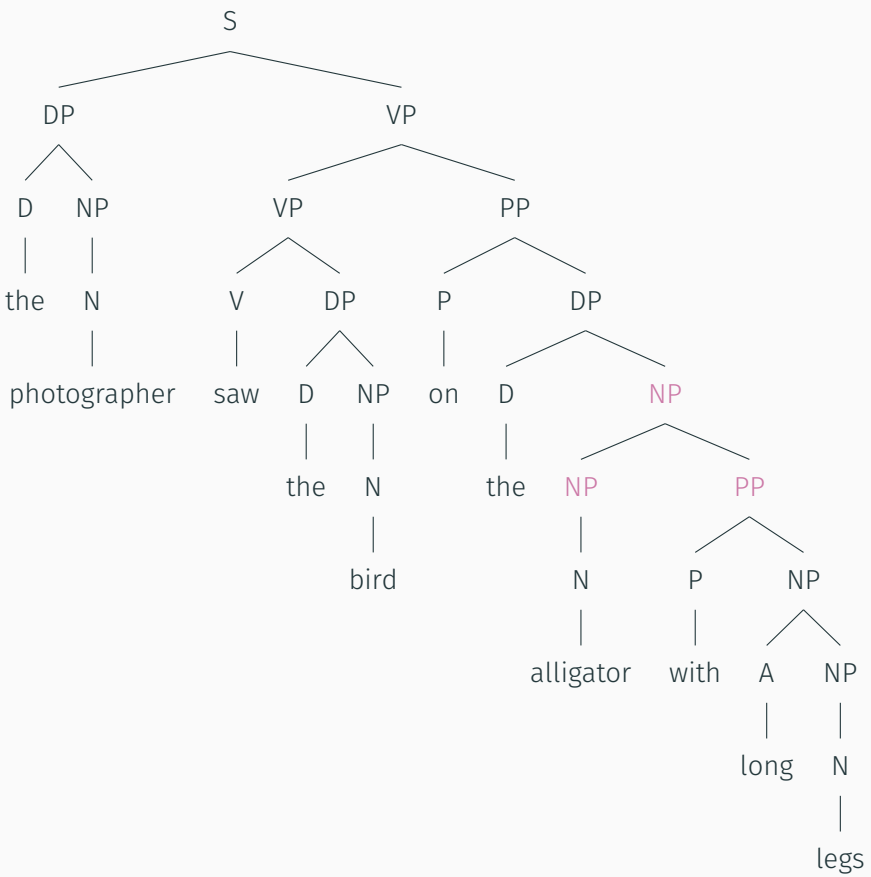
(2) The photographer saw the bird on the alligator with long legs.




- We modify the photographer-on-alligator tree by applying an extra NP $\rightarrow$ NP PP rule right above *alligator*, to modify *alligator* with *with long legs*.

# Structural ambiguity 2': double PP-attachment

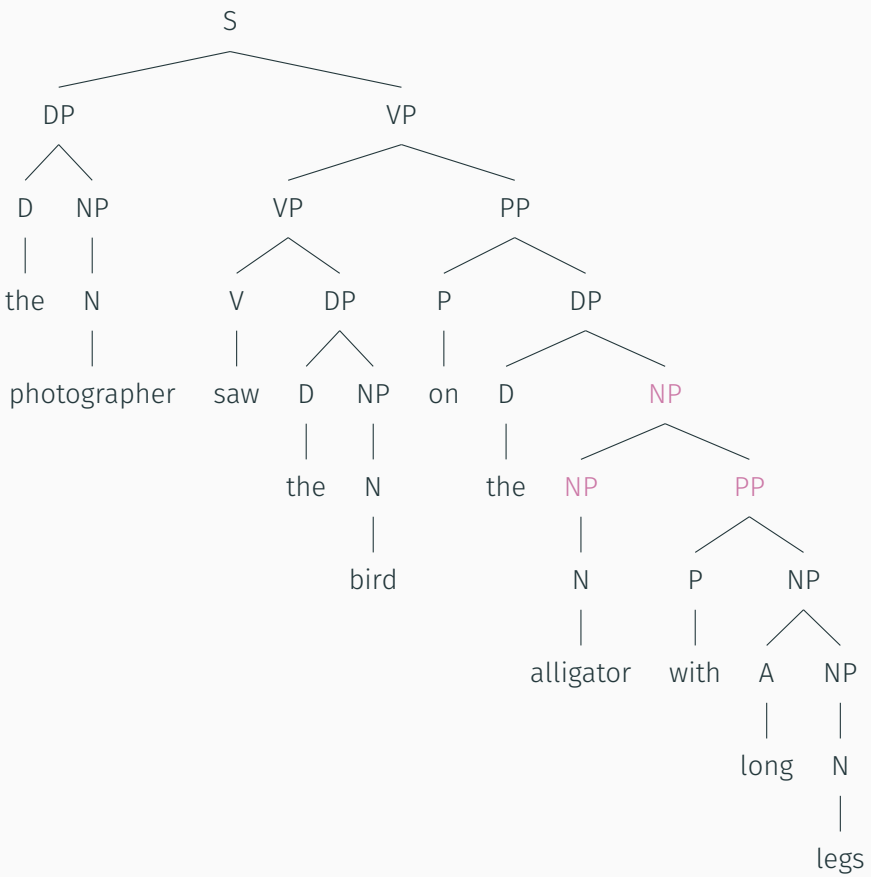
(2) The photographer saw the bird on the alligator with long legs.




- We modify the photographer-on-alligator tree by applying an extra NP $\rightarrow$ NP PP rule right above *alligator*, to modify *alligator* with *with long legs*.
- Reading:

# Structural ambiguity 2': double PP-attachment

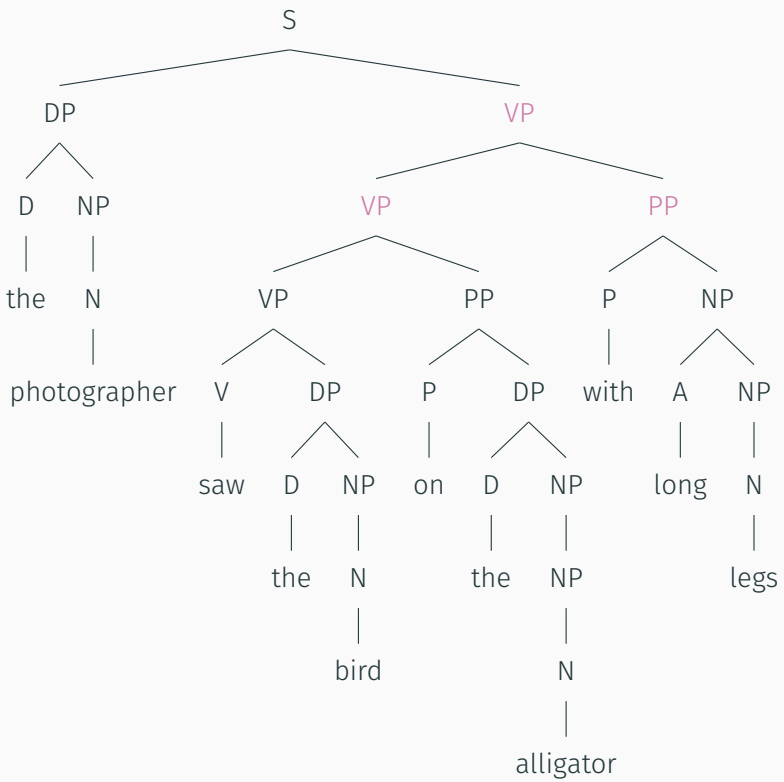
(2) The photographer saw the bird on the alligator with long legs.




- We modify the photographer-on-alligator tree by applying an extra NP $\rightarrow$ NP PP rule right above *alligator*, to modify *alligator* with *with long legs*.
- Reading: The photographer saw a bird and the photographer did so on an alligator with long legs.

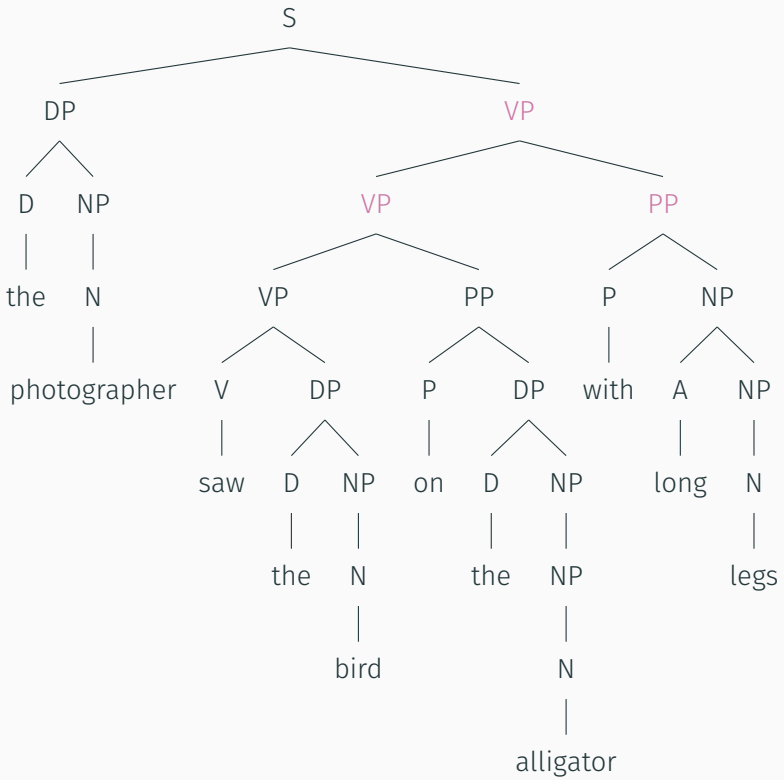
# Structural ambiguity 2': double PP-attachment

(2) The photographer saw the bird on the alligator with long legs.



# Structural ambiguity 2': double PP-attachment

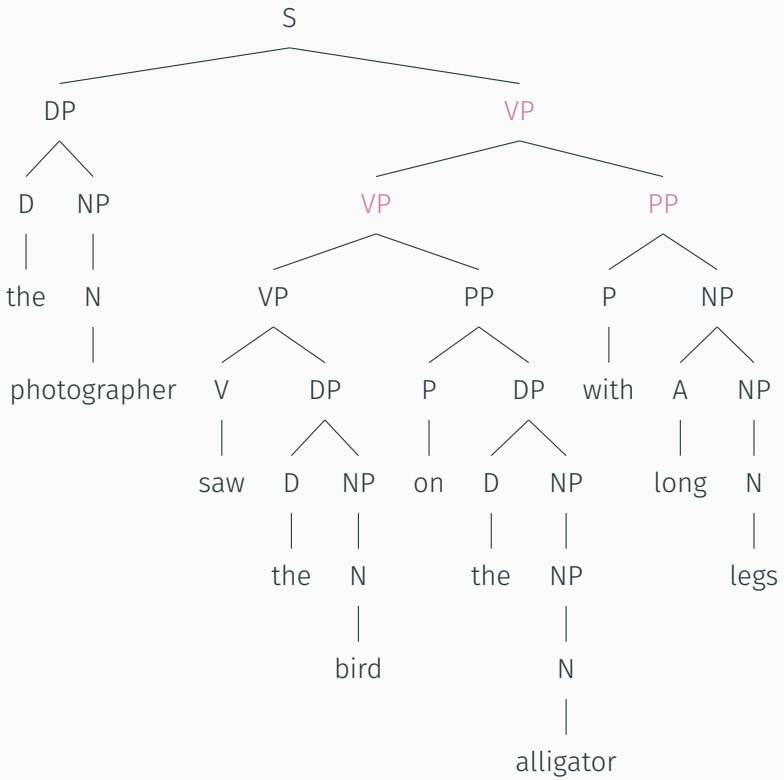
(2) The photographer saw the bird on the alligator with long legs.




- We modify the photographer-on-alligator tree by applying an extra VP $\rightarrow$ VP PP rule to modify *saw* with *with long legs*.

# Structural ambiguity 2': double PP-attachment

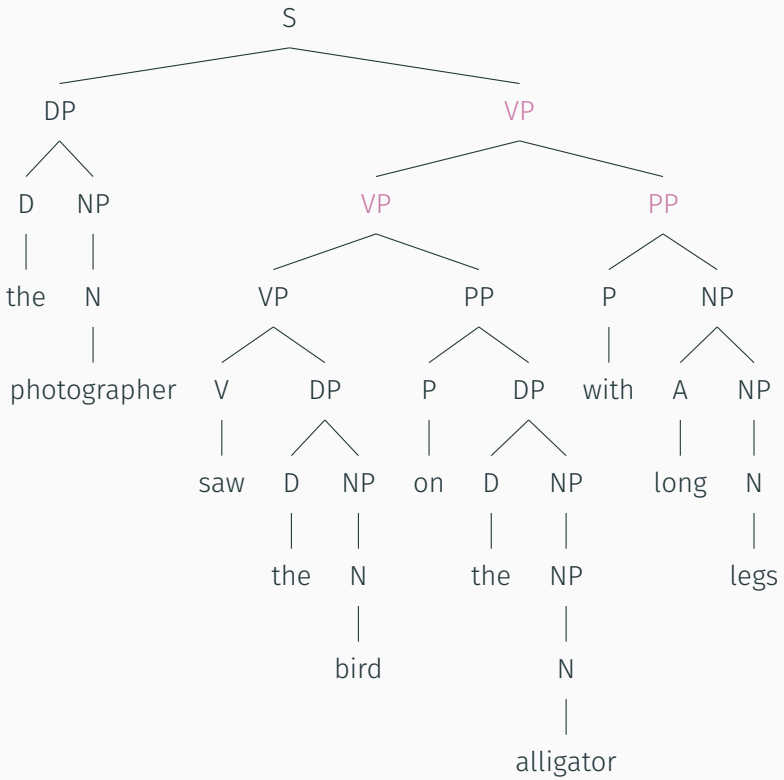
(2) The photographer saw the bird on the alligator with long legs.




- We modify the photographer-on-alligator tree by applying an extra VP $\rightarrow$ VP PP rule to modify *saw* with *with long legs*.
- Reading:

# Structural ambiguity 2': double PP-attachment

(2) The photographer saw the bird on the alligator with long legs.




- We modify the photographer-on-alligator tree by applying an extra VP $\rightarrow$ VP PP rule to modify *saw* with *with long legs*.
- Reading: The photographer saw a bird and the photographer did so on an alligator "using" long legs. Possible but semantically undesired![1]

[1] Why is it a semantic issue? Replace *long legs* by *powerful binoculars*... the reading now makes sense!

# Context-free grammar and structural ambiguities recap

- Context-free grammars primarily generate strings by **calling production rules** (i.e. branching), until all the outputs are terminals.
- But interestingly, keeping track of which production rules were used to generate the final string (and what was their exact timing), sometimes allows us to **derive distinct syntax trees for the same string**–cashing out semantic ambiguities.
- In that sense, context-free grammars have a **strong generative capacity** when it comes to adjective and PP attachment ambiguities.
- We proceed to show that context-free grammar also capture center-embedding constructions, that regexes could not match.

# Context-free grammar and structural ambiguities recap

- Context-free grammars primarily generate strings by **calling production rules** (i.e. branching), until all the outputs are terminals.
- But interestingly, keeping track of which production rules were used to generate the final string (and what was their exact timing), sometimes allows us to **derive distinct syntax trees for the same string**–cashing out semantic ambiguities.
- In that sense, context-free grammars have a **strong generative capacity** when it comes to adjective and PP attachment ambiguities.
- We proceed to show that context-free grammar also capture center-embedding constructions, that regexes could not match.

# Context-free grammar and structural ambiguities recap

- Context-free grammars primarily generate strings by **calling production rules** (i.e. branching), until all the outputs are terminals.
- But interestingly, keeping track of which production rules were used to generate the final string (and what was their exact timing), sometimes allows us to **derive distinct syntax trees for the same string**–cashing out semantic ambiguities.
- In that sense, context-free grammars have a **strong generative capacity** when it comes to adjective and PP attachment ambiguities.
- We proceed to show that context-free grammar also capture center-embedding constructions, that regexes could not match.

# Context-free grammar and structural ambiguities recap

- Context-free grammars primarily generate strings by **calling production rules** (i.e. branching), until all the outputs are terminals.
- But interestingly, keeping track of which production rules were used to generate the final string (and what was their exact timing), sometimes allows us to **derive distinct syntax trees for the same string**–cashing out semantic ambiguities.
- In that sense, context-free grammars have a **strong generative capacity** when it comes to adjective and PP attachment ambiguities.
- We proceed to show that context-free grammar also capture center-embedding constructions, that regexes could not match.

# Center-embedding reminder

(6) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

# Center-embedding reminder

(7) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

# Center-embedding reminder

(8) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

# Center-embedding reminder

(9) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

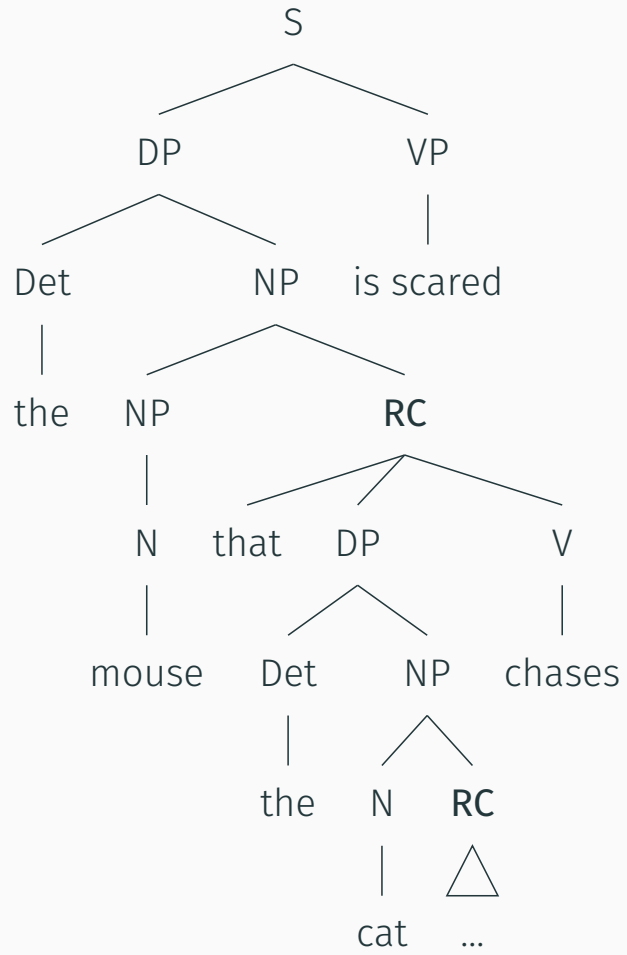
# Center-embedding reminder

(10)
a. The mouse is scared.
b. The mouse [that the cat chases] is scared.
c. The mouse [that the cat [that the dog barks at] chases] is scared.
d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

# Center-embedding: tree

- Recall the center-embedding pattern that regexes and automata could not capture: NP $\underbrace{(\text{that NP})^k(\text{V})^k}_{\text{nested relative clauses}}$ VP, **for any** $k \geq 0$.

- We treat relative clauses (**RCs**) a bit like **complex adjectives**: they may attach to NPs, below the determiner.
- Trick: RCs themselves contain NPs, which themselves may contain other RCs! This makes RCs **recursive**: RCs may introduce new, nested RCs.
- Ultimately, this enables center-embedding.

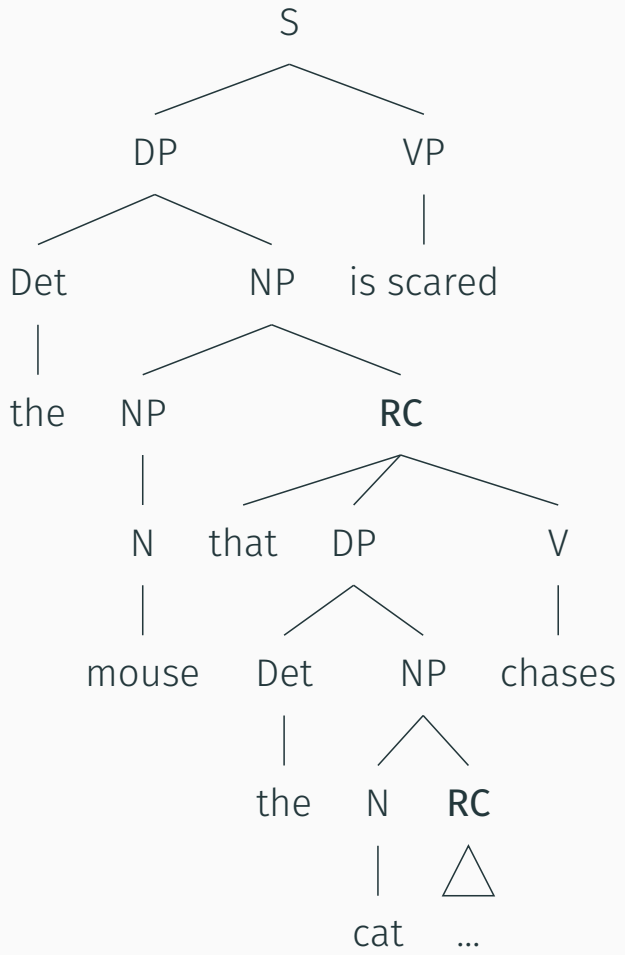


# Center-embedding: tree

- Recall the center-embedding pattern that regexes and automata could not capture: NP $\underbrace{(\text{that NP})^k(\text{V})^k}$ VP, **for any** $k \geq 0$.

nested relative clauses

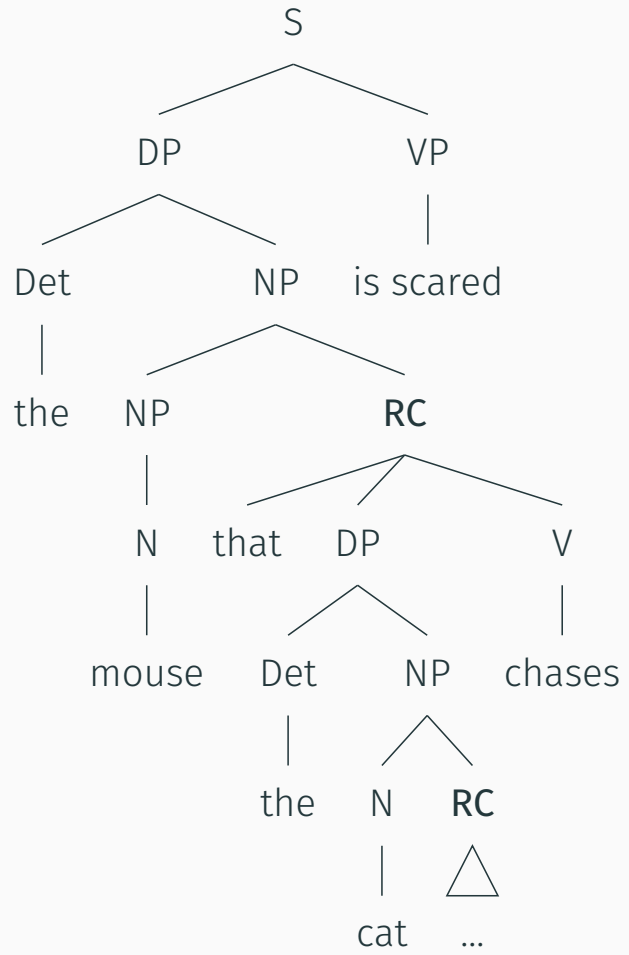
- We treat relative clauses (**RCs**) a bit like **complex adjectives**: they may attach to NPs, below the determiner.
- Trick: RCs themselves contain NPs, which themselves may contain other RCs! This makes RCs **recursive**: RCs may introduce new, nested RCs.
- Ultimately, this enables center-embedding.



# Center-embedding: tree

- Recall the center-embedding pattern that regexes and automata could not capture: NP $\underbrace{(\text{that NP})^k(\text{V})^k}_{\text{nested relative clauses}}$ VP, **for any** $k \geq 0$.

- We treat relative clauses (**RCs**) a bit like **complex adjectives**: they may attach to NPs, below the determiner.
- Trick: RCs themselves contain NPs, which themselves may contain other RCs! This makes RCs **recursive**: RCs may introduce new, nested RCs.
- Ultimately, this enables center-embedding.



# Center-embedding: critical production rules

- Focusing on the nonterminal production rules:

$S \rightarrow DP\ VP$

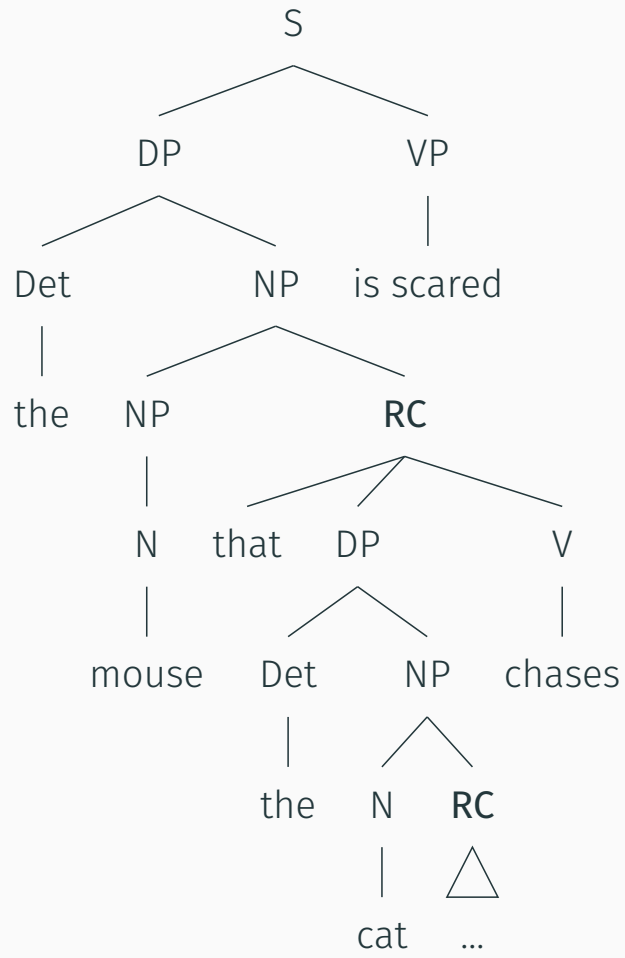
$DP \rightarrow Det\ NP$

$NP \rightarrow NP\ \mathbf{RC}$

$NP \rightarrow \mathbf{N}$

$RC \rightarrow that\ \mathbf{DP}\ V$

- NPs introduce RCs or simply yield a N (crucial base case, for the derivation to stop at some point!)
- RCs in turn introduce DPs, which reintroduce NPs, which may reintroduce RCs!



# Center-embedding: critical production rules

- Focusing on the nonterminal production rules:

$S \rightarrow DP\ VP$

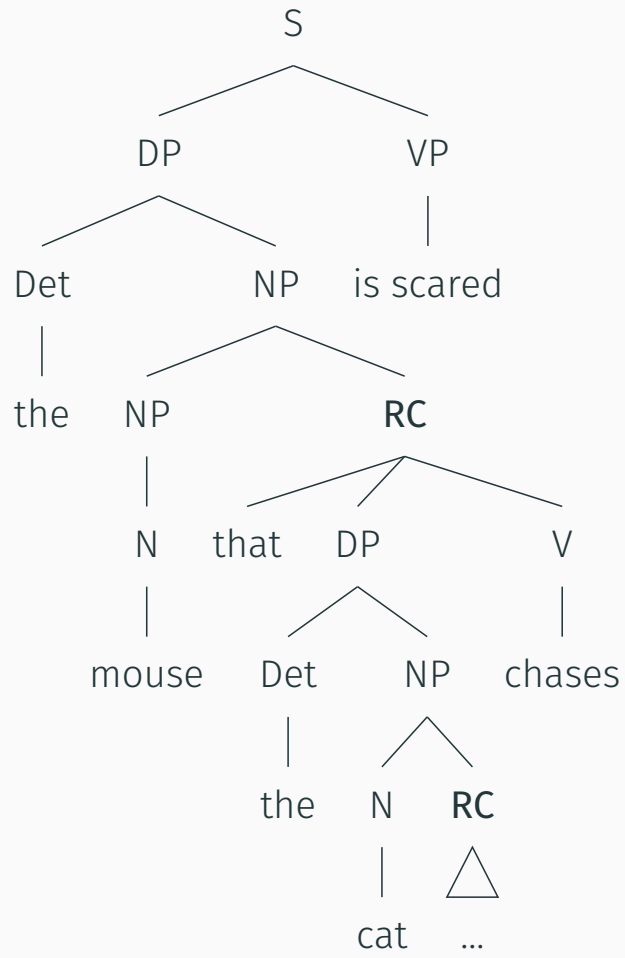
$DP \rightarrow Det\ NP$

$NP \rightarrow NP\ \mathbf{RC}$

$NP \rightarrow \mathbf{N}$

$RC \rightarrow that\ \mathbf{DP}\ V$

- NPs introduce RCs or simply yield a N (crucial base case, for the derivation to stop at some point!)
- RCs in turn introduce DPs, which reintroduce NPs, which may reintroduce RCs!



# Center-embedding: critical production rules

- Focusing on the nonterminal production rules:

$S \rightarrow$ DP VP

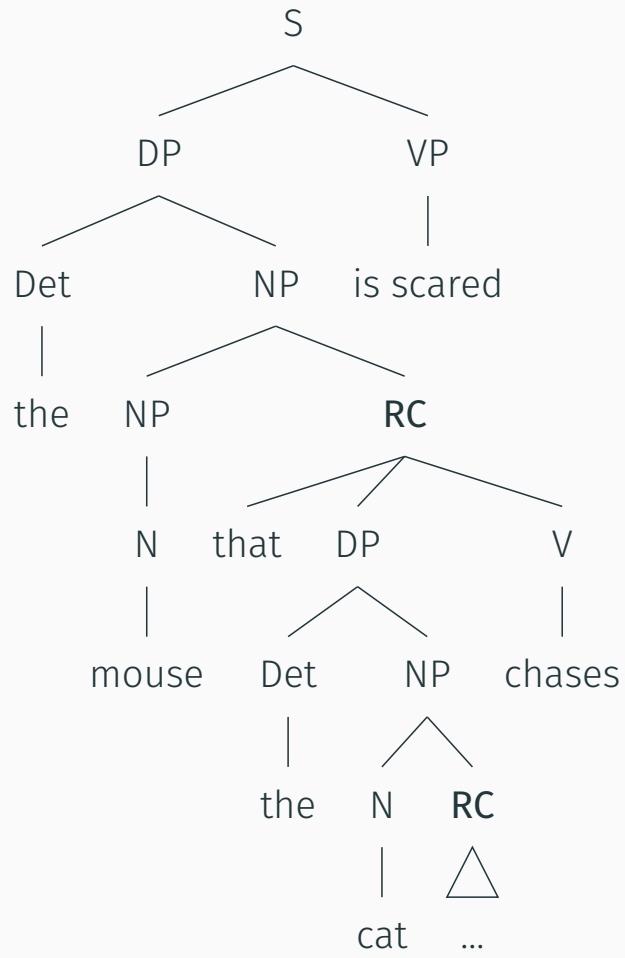
DP $\rightarrow$ Det NP

NP $\rightarrow$ NP **RC**

NP $\rightarrow$ **N**

RC $\rightarrow$ that **DP** V

- NPs introduce RCs or simply yield a N (crucial base case, for the derivation to stop at some point!)
- RCs in turn introduce DPs, which reintroduce NPs, which may reintroduce RCs!



# Center-embedding: critical production rules

- Focusing on the nonterminal production rules:

$S \rightarrow$ DP VP

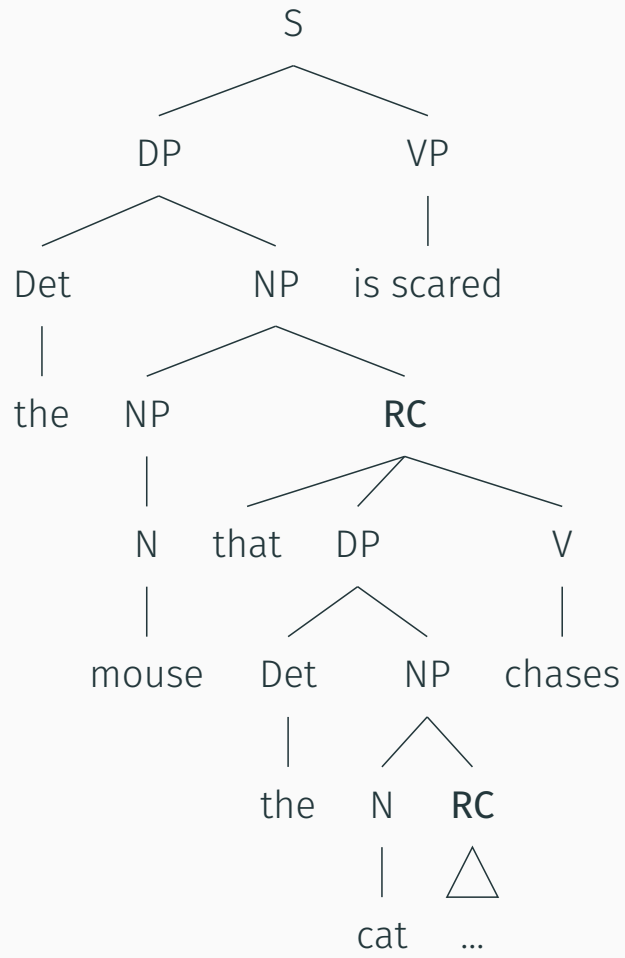
DP $\rightarrow$ Det NP

NP $\rightarrow$ NP **RC**

NP $\rightarrow$ **N**

RC $\rightarrow$ that **DP** V

- NPs introduce RCs or simply yield a N (crucial base case, for the derivation to stop at some point!)
- RCs in turn introduce DPs, which reintroduce NPs, which may reintroduce RCs!



# Challenge: cross-serial dependencies

(11) Swiss German:

…mer em Hans es huus hälfed aastriiche.
…we Hans.DAT the house.ACC help paint.

'… we helped Hans paint the house.'

- In Swiss German, **predicates and their argument can be all jumbled up**! Nevertheless, predicates overtly **case-mark** their arguments, with Dative (in the case of *help*) and Accusative (in the case of *paint*).
- If we wanted to design a context-free grammar keeping track of predicate-argument dependencies (and of the resulting case-marking patterns), we'd struggle in the case of Swiss German, precisely **because the dependencies cross**.
- This pattern is quite rare cross-linguistically; dependencies tend to be nested. But it suggests that natural languages are in fact **more expressive than context-free grammars**.

# Challenge: cross-serial dependencies

(12) Swiss German:

…mer em Hans es huus hälfed aastriiche.
…we Hans.DAT the house.ACC help paint.

'… we helped Hans paint the house.'

- In Swiss German, **predicates and their argument can be all jumbled up**! Nevertheless, predicates overtly **case-mark** their arguments, with Dative (in the case of *help*) and Accusative (in the case of *paint*).
- If we wanted to design a context-free grammar keeping track of predicate-argument dependencies (and of the resulting case-marking patterns), we'd struggle in the case of Swiss German, precisely **because the dependencies cross**.
- This pattern is quite rare cross-linguistically; dependencies tend to be nested. But it suggests that natural languages are in fact **more expressive than context-free grammars**.

# Challenge: cross-serial dependencies

(13) Swiss German:

…mer em Hans es huus hälfed aastriiche.
…we Hans.DAT the house.ACC help paint.

'… we helped Hans paint the house.'

- In Swiss German, **predicates and their argument can be all jumbled up**! Nevertheless, predicates overtly **case-mark** their arguments, with Dative (in the case of *help*) and Accusative (in the case of *paint*).
- If we wanted to design a context-free grammar keeping track of predicate-argument dependencies (and of the resulting case-marking patterns), we'd struggle in the case of Swiss German, precisely **because the dependencies cross**.
- This pattern is quite rare cross-linguistically; dependencies tend to be nested. But it suggests that natural languages are in fact **more expressive than context-free grammars**.

# Challenge: cross-serial dependencies

(14) Swiss German:

…mer em Hans es huus hälfed aastriiche.
…we Hans.DAT the house.ACC help paint.

'… we helped Hans paint the house.'

- In Swiss German, **predicates and their argument can be all jumbled up**! Nevertheless, predicates overtly **case-mark** their arguments, with Dative (in the case of *help*) and Accusative (in the case of *paint*).
- If we wanted to design a context-free grammar keeping track of predicate-argument dependencies (and of the resulting case-marking patterns), we'd struggle in the case of Swiss German, precisely **because the dependencies cross**.
- This pattern is quite rare cross-linguistically; dependencies tend to be nested. But it suggests that natural languages are in fact **more expressive than context-free grammars**.

# Challenge: cross-serial dependencies

(15) Swiss German:

…mer em Hans es huus hälfed aastriiche.
…we Hans.DAT the house.ACC help paint.

'… we helped Hans paint the house.'

- In Swiss German, **predicates and their argument can be all jumbled up**! Nevertheless, predicates overtly **case-mark** their arguments, with Dative (in the case of *help*) and Accusative (in the case of *paint*).
- If we wanted to design a context-free grammar keeping track of predicate-argument dependencies (and of the resulting case-marking patterns), we'd struggle in the case of Swiss German, precisely **because the dependencies cross**.
- This pattern is quite rare cross-linguistically; dependencies tend to be nested. But it suggests that natural languages are in fact **more expressive than context-free grammars**.

# Lab

https://colab.research.google.com/drive/1Hh5Xz3awu5_EDNQPXdql-sTnZkPAfuizr?usp=sharing